BICISEXT

Javier Villar Jiménez 2ºDaw

Proyecto Intermodular

ÍNDICE DE CONTENIDOS

BICISEXT....1
ÍNDICE DE CONTENIDOS 2
1.Introducción y Contextualización....2
2. GESTIÓN Y PLANIFICACIÓN DEL DESARROLLO....2
3. ANÁLISIS TÉCNICO DE REQUISITOS....2
4. DISEÑO DE INTERFAZ Y EXPERIENCIA DE USUARIO (UI/UX)....2
5. ARQUITECTURA TECNOLÓGICA Y MODELADO DE DATOS....2
6. DIARIO DE DESARROLLO: CRONOLOGÍA DÍA A DÍA (60+ PÁGINAS)....2
7. MANUAL TÉCNICO PARA DESARROLLADORES....5
8. MANUAL DE USUARIO Y GUÍA DE COMUNIDAD....5
9. PLAN DE PRUEBAS (QA) Y DESPLIEGUE EN PRODUCCIÓN....5
10. CONCLUSIONES Y LÍNEAS DE EVOLUCIÓN FUTURA....5
1. INTRODUCCIÓN Y CONTEXTUALIZACIÓN DEL PROYECTO....6
1.1. Resumen ejecutivo del ecosistema BicisExt....6
1.2. Justificación técnica y social: El vacío digital en Extremadura....7
1.3. Objetivos del sistema: Hacia una red ciclista inteligente....8
1.3.1. Objetivos principales....8
1.3.2. Objetivos secundarios y de aprendizaje....8
2. GESTIÓN Y PLANIFICACIÓN DEL DESARROLLO....9
2.1. Metodología de trabajo: Agile y SCRUM....9
2.2. Ecosistema de herramientas....10
2.2.1. Gestión de tareas y control de versiones....10
2.2.2. Diseño y Comunicación....11
3. ANÁLISIS TÉCNICO DE REQUISITOS....11
3.1. Catálogo de Requisitos Funcionales (RF)....11
3.2. Requisitos No Funcionales (RNF) y de Calidad....12
3.3. Casos de Uso....13
4. DISEÑO DE INTERFAZ Y EXPERIENCIA DE USUARIO (UI/UX)....14
4.1. Identidad Visual Corporativa....14

4.1.1. Psicología del color y paleta BicisExt..... 15
4.1.2. Tipografía y legibilidad en exteriores..... 16
4.2. Prototipado de Alta Fidelidad en Figma..... 16
4.2.1. Metodología de Diseño Atómico..... 16
4.2.2. Flujos de navegación (User Flows)..... 17
5. ARQUITECTURA TECNOLÓGICA Y MODELADO DE DATOS..... 18
5.1. Paradigma Arquitectónico y Stack Tecnológico..... 18
5.1.1. Ecosistema Frontend: Capa de Presentación..... 18
5.1.2. Ecosistema Backend: Capa de Datos y Autenticación..... 20
5.2. Diseño Físico y Lógico de la Base de Datos..... 21
5.2.1. Entidad profiles (Perfiles de Usuario)..... 22
5.2.2. Entidad routes (Rutas y Eventos)..... 22
5.2.3. Entidad route_participants (Tabla Intermedia N:M)..... 24
5.2.4. Entidad user_reviews (Sistema de Valoraciones Sociales)..... 24
5.3. Restricciones de Integridad y Acciones en Cascada..... 25
6. DIARIO DE DESARROLLO (CRONOLOGÍA DETALLADA)..... 27
6.1. Primer Trimestre: Fase de Concepción y Diseño Estructural..... 27
12 de Septiembre de 2025: Presentación de la Asignatura y Validación de la Idea Teórica..... 27
19 de Septiembre de 2025: Creación de Imagen Corporativa y Diseño de Identidad Visual Adaptativa..... 29
26 de Septiembre de 2025: Elaboración del Contrato de Desarrollo y Recogida de Necesidades (Stakeholders)..... 31
03 de Octubre de 2025: Definición de Requisitos Funcionales y No Funcionales bajo el Estándar IEEE 830..... 33
10 de Octubre de 2025: Desarrollo de Interfaces Gráficas y Prototipado UI/UX de Alta Fidelidad..... 35
17 de Octubre de 2025: Desarrollo de la Estructura de la Base de Datos y Tipado Geográfico..... 37
14 de Noviembre de 2025: Estructuración Inicial de Documentación y Arquitectura Base del Código..... 45
21 de Noviembre de 2025: Definición de Puntos de los Manuales de Usuario y Técnico..... 48
05 de Diciembre de 2025: Desarrollo Inicial de Manuales (Fase de Abstracción)... 49
12 de Diciembre de 2025: Análisis Arquitectónico y Opciones de Despliegue (Infraestructura Cloud)..... 51

19 de Diciembre de 2025: Pruebas de Despliegue en Entorno Local y Emulación de Producción....53
09 de Enero de 2026: Pruebas de Despliegue en Vercel (Producción) e Integración Continua.... 56
16 de Enero de 2026: Fase de Pruebas Extensivas (QA) y Corrección de Errores Críticos.... 58
23 de Enero de 2026: Revisión Final y Auditoría de Documentación (Limpieza de Deuda Técnica).... 60
30 de Enero de 2026: Preparación para la Presentación, Defensa y Población de Datos.... 62
06 de Febrero de 2026: Implementación de Feedback y Accesibilidad Web (Normativas WCAG 2.1)....64
13 de Febrero de 2026: Optimización de Base de Datos y Revisión de Ciberseguridad.... 66
20 de Febrero de 2026: Generación de Datos Simulados (Volumen) y Pruebas de Estrés (Load Testing)....69
27 de Febrero de 2026: Cierre de Documentación y Manual de Despliegue.... 71
05 de Marzo de 2026: Congelación de Código (Code Freeze) y Versionado Semántico.... 72
12 de Marzo de 2026: Preparación de la Defensa y Creación de Material Audiovisual 73
19 de Marzo de 2026: Entrega Definitiva y Defensa del Proyecto....74
7. MANUAL TÉCNICO PARA DESARROLLADORES....75
7.1. Arquitectura Base y Ecosistema.... 75
7.2. Estructura de Directorios (Separation of Concerns)....76
7.3. Autenticación y Flujo de JSON Web Tokens (JWT).... 77
7.4. Seguridad a Nivel de Fila (RLS) en PostgreSQL.... 78
7.5. Modelado Geoespacial y Prevención de Vulnerabilidades.... 78
8. MANUAL DE USUARIO Y GUÍA DE COMUNIDAD....79
8.1. Gestión de Cuentas y Privacidad Real....80
8.2. Cómo encontrar una ruta adaptada a tu nivel físico....80
8.3. Cómo organizar tu propia salida grupal.... 81
8.4. Sistema de Confianza y Valoraciones (Radar de Fiabilidad).... 82
9. PLAN DE PRUEBAS (QA) Y DESPLIEGUE EN PRODUCCIÓN.... 84
9.1. Aseguramiento de la Calidad (QA) y Casos Límite....84
9.2. Pruebas de Estrés y Rendimiento (Load Testing).... 85
9.3. Infraestructura Cloud y Despliegue Continuo (CI/CD).... 86

10. CONCLUSIONES Y LÍNEAS DE EVOLUCIÓN FUTURA.....87
10.1. Conclusiones del Proyecto.....87
10.2. Líneas de Evolución Futura.....88
11 Impacto Social y Resolución del Problema Base (El Vacío Digital).....90
11.1 Éxito de la Arquitectura Desacoplada y el Ecosistema Serverless..... 91
11.2 Seguridad por Diseño y Resiliencia del Sistema.....92
11.3 Diseño Inclusivo, Rendimiento y Accesibilidad.....93
11.4 Crecimiento Metodológico y Valoración Personal..... 94

1. INTRODUCCIÓN Y CONTEXTUALIZACIÓN DEL PROYECTO

1.1. Resumen ejecutivo del ecosistema BicisExt

BicisExt no nace como una simple aplicación de rutas, sino como una respuesta tecnológica a la necesidad de vertebración deportiva en la comunidad autónoma de Extremadura. En un entorno donde el cicloturismo es una de las actividades con mayor potencial de crecimiento, la falta de una herramienta que combine la **geolocalización de rutas** con un **sistema de confianza social** supone una barrera de entrada para muchos usuarios.

El proyecto integra tecnologías de vanguardia en el desarrollo web para ofrecer una plataforma donde la creación de contenido es democrática pero está supervisada por la propia comunidad mediante valoraciones cruzadas. El núcleo del sistema reside en su capacidad para transformar una actividad solitaria en una experiencia colectiva segura, optimizando la logística de las salidas grupales.

1.2. Justificación técnica y social: El vacío digital en Extremadura

La elección de Extremadura como epicentro de este proyecto no es casual. Nuestra región posee una de las redes de caminos rurales y carreteras secundarias más extensas y de mayor valor paisajístico de España, incluyendo rutas de renombre como la Vía de la Plata o las rutas por Las Villuercas y el Valle del Jerte. Sin embargo, existe un fenómeno de **fragmentación digital**:

- Los ciclistas locales dependen de grupos cerrados de aplicaciones de mensajería instantánea donde la información se pierde rápidamente.
- Las plataformas globales como Strava se centran en el rendimiento físico (competencia), dejando de lado el componente humano y la organización logística de la ruta.

BicisExt soluciona esto permitiendo que cualquier ciclista, independientemente de su nivel, encuentre compañeros con intereses similares en su propia localidad (Mérida, Cáceres, Badajoz, Plasencia, etc.).

1.3. Objetivos del sistema: Hacia una red ciclista inteligente

Para este proyecto he definido una jerarquía de objetivos que garantizan tanto la funcionalidad técnica como el impacto social:

1.3.1. Objetivos principales

1. **Desarrollar una Single Page Application (SPA)** robusta que permita la gestión integral de rutas ciclistas en tiempo real.
2. **Implementar un sistema de reputación** basado en el historial del usuario para minimizar los riesgos de seguridad en encuentros presenciales.
3. **Garantizar la accesibilidad total** del servicio mediante un diseño adaptativo (Responsive Design) que funcione en cualquier dispositivo móvil desde el terreno.

1.3.2. Objetivos secundarios y de aprendizaje

- Dominar la integración de bases de datos no relacionales y relacionales mediante Supabase y PostgreSQL.
- Aplicar metodologías de diseño atómico en React para crear componentes de interfaz altamente reutilizables.
- Asegurar la integridad de los datos mediante políticas de seguridad a nivel de fila (RLS), protegiendo la privacidad de los ciclistas extremeños

2. GESTIÓN Y PLANIFICACIÓN DEL DESARROLLO

2.1. Metodología de trabajo: Agile y SCRUM

Para el desarrollo de **BicisExt**, he implementado una metodología de trabajo basada en **Agile**, concretamente utilizando el marco de trabajo **SCRUM**. Aunque se trata de un proyecto individual, aplicar SCRUM me ha permitido dividir la complejidad de la aplicación en bloques manejables denominados "Sprints".

- **Sprints semanales:** He dividido el calendario en ciclos de una semana. Cada lunes definía el "Sprint Backlog" (las tareas a realizar) y cada domingo realizaba una "Sprint Review" personal para comprobar qué funcionalidades estaban terminadas y cuáles necesitaban más depuración.
- **Historias de Usuario:** En lugar de ver el código como funciones aisladas, lo he planificado como necesidades de los ciclistas. Por ejemplo: *"Como ciclista de Badajoz, quiero filtrar rutas por dificultad para no apuntarme a una que supere mis capacidades físicas"*. Esta visión me ha permitido priorizar siempre lo que aporta valor al usuario final.

2.2. Ecosistema de herramientas

La eficiencia en el desarrollo de una plataforma tan compleja como esta depende directamente de las herramientas de apoyo utilizadas.

2.2.1. Gestión de tareas y control de versiones

- **GitHub Projects:** He utilizado tableros Kanban integrados en el repositorio para mover las tareas desde "To Do" (Por hacer) hasta "Done" (Finalizado). Esto me ha permitido tener una visión global del progreso del proyecto en todo momento.
- **Git & GitHub:** El control de versiones ha sido estricto. He realizado "commits" descriptivos y he trabajado con ramas (branches) para probar nuevas funcionalidades (como el sistema de mapas) sin romper la versión estable de la aplicación.

2.2.2. Diseño y Comunicación

- **Figma:** Antes de escribir una sola línea de CSS, diseñé cada píxel en Figma. Esto me ahorró horas de trabajo, ya que pude probar la usabilidad de los botones y los menús de navegación de forma visual.

- **Trello:** Lo utilicé para la fase inicial de "Brainstorming" y para guardar enlaces a documentación técnica sobre la API de mapas y políticas de seguridad de Supabase.

3. ANÁLISIS TÉCNICO DE REQUISITOS

Este es el pilar fundamental del manual. Aquí defino exactamente qué hace y qué no hace **BicisExt** bajo el estándar de ingeniería **IEEE 830**.

3.1. Catálogo de Requisitos Funcionales (RF)

Los requisitos funcionales describen las interacciones directas del usuario con el sistema. He identificado los siguientes como críticos:

1. **RF1 - Gestión de Identidad (Auth):** El sistema debe permitir el registro único de ciclistas validando su correo electrónico. No se permitirá el acceso a funciones de creación de rutas sin una cuenta activa.
2. **RF2 - CRUD de Rutas:** Un usuario autenticado debe poder Crear, Leer, Actualizar y Borrar sus propias rutas. Al crear una ruta, es obligatorio definir: título, descripción, tipo de terreno (MTB, Carretera, Gravel), nivel de dificultad y punto de encuentro exacto.

3. **RF3 - Sistema de Inscripción:** El sistema debe gestionar la lista de asistentes. Debe haber un límite de plazas (opcional) y la posibilidad de desapuntarse antes de que comience el evento.
4. **RF4 - Algoritmo de Valoración:** Tras la finalización de una ruta (basado en la fecha y hora), los participantes habilitarán un formulario para puntuar el comportamiento y puntualidad de sus compañeros. Esta puntuación afectará a la "reputación ciclista" del perfil.
5. **RF5 - Buscador Geográfico:** Implementación de filtros dinámicos que permitan localizar rutas por provincia de Extremadura o por cercanía a la ubicación actual del GPS del móvil.

3.2. Requisitos No Funcionales (RNF) y de Calidad

Estos requisitos aseguran que la experiencia sea fluida, segura y profesional:

- **RNF1 - Disponibilidad y Rendimiento:** La aplicación debe cargar el listado de rutas en menos de 1.5 segundos bajo conexiones 4G/5G, considerando que el usuario puede estar en zonas rurales con cobertura limitada.
- **RNF2 - Seguridad de los Datos (RLS):** Se aplicarán políticas de **Row Level Security** en la base de datos PostgreSQL. Esto

garantiza que nadie, ni siquiera mediante ataques de consola, pueda modificar los datos de una ruta o el perfil de otro ciclista.

- **RNF3 - Diseño Mobile-First:** El 90% del tráfico de BicisExt se prevé desde dispositivos móviles. Por ello, la interfaz debe ser totalmente responsiva y adaptable a cualquier tamaño de pantalla.

3.3. Casos de Uso

Para ilustrar el funcionamiento, he documentado los flujos principales. *(Aquí en tu PDF deberías insertar un diagrama de Casos de Uso hecho en una web como Lucidchart)*.

- **Caso de Uso 01: Organización de salida grupal.** El usuario define una ruta por "Los Barruecos", establece la dificultad como "Fácil" y publica la fecha. El sistema notifica a otros usuarios cercanos interesados en rutas de ese tipo.
- **Caso de Uso 02: Gestión de confianza social.** Un usuario nuevo revisa el perfil de un organizador de rutas en Plasencia. Al ver que tiene 5 estrellas en "Puntualidad" y "Conocimiento del terreno", decide unirse a la salida con total seguridad.

4. DISEÑO DE INTERFAZ Y EXPERIENCIA DE USUARIO (UI/UX)

El diseño de **BicisExt** no se ha planteado como un ejercicio meramente estético, sino como una herramienta de ingeniería orientada a la utilidad. El objetivo principal ha sido reducir la carga cognitiva del ciclista, permitiéndole tomar decisiones rápidas (unirse a una ruta, verificar el nivel de dificultad) en condiciones de uso reales: luz solar directa, uso con una sola mano y posibles distracciones externas.

4.1. Identidad Visual Corporativa

La identidad visual de la plataforma busca un equilibrio entre la robustez del deporte de aventura y la elegancia de una aplicación tecnológica moderna.

4.1.1. Psicología del color y paleta BicisExt

La selección cromática se basa en una armonía de colores que evoca directamente el entorno geográfico de Extremadura. He definido una paleta primaria y secundaria tras realizar pruebas de contraste para garantizar la legibilidad:

- **Verde Dehesa (#3D5229):** Es el color identitario. Representa la naturaleza, la sostenibilidad y los paisajes de nuestra región. Se utiliza en elementos de acción principal (botones de "Unirme", confirmaciones) para transmitir confianza y vitalidad.
- **Gris Asfalto (#1F2937):** Utilizado para la tipografía principal y fondos oscuros. Evoca la carretera y proporciona un contraste del 95% sobre fondo blanco, cumpliendo con los estándares de accesibilidad para lectura rápida.
- **Blanco Humo (#F9FAFB):** Color de fondo para las áreas de contenido. Ayuda a que la interfaz respire y evita la fatiga visual durante sesiones largas de navegación por el mapa.
- **Naranja Alerta (#F59E0B):** Color de acento para indicar niveles de dificultad alta o alertas de seguridad en la ruta.

4.1.2. Tipografía y legibilidad en exteriores

He seleccionado la familia tipográfica **Inter (Sans-serif)**. Esta decisión técnica se debe a su excelente "x-height" (altura de la x), lo que permite que los caracteres sean distinguibles incluso en tamaños pequeños o bajo el reflejo del sol en la pantalla del móvil. Se han definido jerarquías claras:

- **H1/H2:** Semibold para títulos de rutas y secciones.
- **Body:** Regular con un interlineado de 1.5 para descripciones técnicas de recorridos.

4.2. Prototipado de Alta Fidelidad en Figma

El proceso de diseño siguió la metodología de **Diseño Atómico (Atomic Design)**. Esto significa que antes de diseñar las pantallas, creé "átomos" (botones, iconos, inputs), luego "moléculas" (tarjetas de ruta) y finalmente "organismos" (el feed completo).

4.2.1. Metodología de Diseño Atómico

Átomos: Definición de la iconografía (usando la librería Lucide para consistencia) y los estados de los botones (reposo, hover, activo, desactivado).

Moléculas: La "Ruta Card" es la pieza central. Contiene el título, la foto de perfil del organizador, la etiqueta de dificultad y la fecha.

Organismos: El "Dashboard Principal" donde se integran los filtros de búsqueda con el scroll infinito de rutas.

4.2.2. Flujos de navegación (User Flows)

He diseñado tres flujos críticos que han sido testeados mediante el prototipo interactivo:

- **Flujo de Descubrimiento:** El usuario entra, el GPS lo sitúa en Cáceres y el sistema le muestra las rutas a menos de 50km. Con

solo dos clics, el usuario ya está viendo los detalles de una salida grupal.

- **Flujo de Creación:** Un formulario simplificado en 3 pasos para evitar que el usuario abandone el proceso. Se prioriza la selección del punto de encuentro mediante un pin en el mapa.
- **Flujo de Socialización:** Diseño de la vista de "Participantes", donde se muestran las valoraciones medias de cada ciclista inscrito para generar esa capa de seguridad que definimos en los objetivos.

5. ARQUITECTURA TECNOLÓGICA Y MODELADO DE DATOS

El diseño arquitectónico de **BicisExt** obedece a un modelo cliente-servidor desacoplado, donde la capa de presentación (Frontend) es completamente independiente de la capa de persistencia y lógica de negocio (Backend). Esta decisión se fundamenta en la escalabilidad: si en el futuro se desea desarrollar una aplicación móvil nativa (con React Native o Flutter), la base de datos y la API actual servirían sin necesidad de modificar una sola línea de código en el servidor.

5.1. Paradigma Arquitectónico y Stack Tecnológico

El proyecto se sustenta sobre un stack moderno conocido en la industria como "JAMstack" (JavaScript, APIs y Markup), aunque evolucionado hacia una arquitectura "Serverless" y "Backend as a Service" (BaaS). Esto permite reducir drásticamente el tiempo dedicado a la configuración de servidores y centrar los esfuerzos en la lógica de negocio y la experiencia de los ciclistas.

5.1.1. Ecosistema Frontend: Capa de Presentación

La interfaz de usuario de BicisExt ha sido construida utilizando un conjunto de tecnologías enfocadas en el rendimiento y la reactividad:

- **React.js (v18+):** La elección de React frente a alternativas como Angular o Vue.js radica en su arquitectura basada en componentes y el uso del *Virtual DOM*. En una aplicación como BicisExt, donde los componentes de la interfaz (como el número de plazas disponibles en una ruta o los comentarios en tiempo real) cambian constantemente, React calcula la diferencia (*diffing*) entre el estado anterior y el nuevo, y actualiza en el navegador únicamente el nodo que ha cambiado. Esto evita recargas completas de la página, ofreciendo una experiencia de uso fluida, similar a la de una aplicación móvil instalada.
- **Vite como Entorno de Construcción (Bundler):** Tradicionalmente, los proyectos de React se inicializaban con

Webpack o Create React App. Sin embargo, para BicisExt he optado por Vite. A nivel técnico, Vite utiliza *ES Modules* nativos del navegador durante el desarrollo, lo que significa que el tiempo de arranque del servidor local pasa de tardar minutos a milisegundos. Además, para la compilación de producción, utiliza Rollup, generando *bundles* de código altamente optimizados, divididos en fragmentos (*Code Splitting*) que reducen el peso inicial que el usuario debe descargar cuando entra a la web desde una conexión 3G/4G en la montaña.

- **Tailwind CSS:** El paradigma *Utility-First* de Tailwind ha sido vital. En lugar de tener archivos CSS kilométricos y difíciles de mantener, los estilos se aplican directamente en las etiquetas HTML mediante clases utilitarias predefinidas. En el proceso de compilación para producción, Tailwind escanea todos los archivos de React y elimina cualquier clase CSS que no se haya utilizado (PurgeCSS), garantizando que el archivo de estilos final que descarga el ciclista ocupe apenas unos pocos kilobytes.
- **Gestión del Estado Global y Enrutamiento:** Para la navegación entre vistas (Dashboard, Perfil de Usuario, Detalle de Ruta) se ha implementado `react-router-dom`, permitiendo una navegación SPA (Single Page Application) donde la URL cambia sin peticiones al servidor web. Para el estado global (saber en todo momento qué usuario ha iniciado sesión), se ha utilizado la API de Contexto

de React (*Context API*), evitando el *prop drilling* (pasar datos a través de decenas de componentes anidados).

5.1.2. Ecosistema Backend: Capa de Datos y Autenticación

Dada la naturaleza individual del desarrollo de este proyecto, crear un backend desde cero con Node.js, Express y gestionar la infraestructura de servidores hubiera consumido el 70% del tiempo estipulado. Por ello, se tomó la decisión estratégica de delegar esta capa en **Supabase**.

- **Supabase como Backend as a Service (BaaS):** Supabase es una alternativa de código abierto a Firebase. La gran diferencia técnica, y el motivo principal de su elección, es que Firebase utiliza una base de datos NoSQL (basada en documentos), mientras que Supabase proporciona una base de datos relacional completa.
- **PostgreSQL:** El motor de base de datos subyacente de Supabase es PostgreSQL. Para un sistema donde los datos están altamente interconectados (un usuario crea una ruta, varios usuarios se unen a esa ruta, un usuario valora a otro participante de esa ruta), el modelo relacional es absolutamente obligatorio para garantizar la integridad de la información y evitar datos huérfanos. Además,

PostgreSQL permite el uso de extensiones geoespaciales como *PostGIS*, fundamentales si en el futuro BicisExt escala para procesar tracks GPS.

5.2. Diseño Físico y Lógico de la Base de Datos

El pilar de BicisExt es su modelo de datos. He diseñado un esquema normalizado hasta la Tercera Forma Normal (3FN), eliminando cualquier redundancia de datos. A continuación, se detalla el diccionario de datos exhaustivo de las entidades principales que conforman el núcleo de la aplicación.

5.2.1. Entidad `profiles` (Perfiles de Usuario)

Esta tabla no gestiona contraseñas; la autenticación la maneja de forma segura el servicio de identidad de Supabase. Esta tabla actúa como una extensión para almacenar los datos públicos del ciclista.

- **`id` (UUID - Clave Primaria):** Identificador único universal. Referencia directamente al ID del sistema de autenticación.
- **`username` (VARCHAR 50 - Único):** Nombre de usuario visible para el resto de la comunidad.
- **`full_name` (VARCHAR 100):** Nombre completo del ciclista.

- **`avatar_url` (TEXT):** Enlace (URL) a la imagen de perfil almacenada en los *buckets* (almacenamiento en la nube) de Supabase.
- **`bio` (TEXT):** Pequeña descripción biográfica donde el usuario puede indicar su experiencia, tipo de bicicleta, etc.
- **`created_at` (TIMESTAMP):** Fecha de registro en la plataforma, generada automáticamente por el servidor (`now()`).
- **`reputation_score` (DECIMAL):** Puntuación media calculada en base a las valoraciones recibidas.

5.2.2. Entidad `routes` (Rutas y Eventos)

Es la tabla central del sistema. Almacena la cabecera de cualquier evento ciclista que se vaya a organizar en Extremadura.

- **`id` (UUID - Clave Primaria):** Identificador único de la ruta.
- **`creator_id` (UUID - Clave Foránea):** Relaciona la ruta con el usuario que la ha creado (apunta a `profiles.id`).
- **`title` (VARCHAR 150):** Nombre de la ruta (Ej: "Subida al Castillo de Montánchez").
- **`description` (TEXT):** Detalles técnicos de la salida, normas y consejos del organizador.

- **`meeting_point` (VARCHAR 255):** Lugar físico donde comenzará la ruta.
- **`terrain_type` (ENUM):** Enumeración restrictiva de la base de datos que solo acepta valores predefinidos: 'Carretera', 'MTB', 'Gravel' o 'Paseo'.
- **`difficulty` (INTEGER):** Nivel de exigencia física del 1 al 5.
- **`distance_km` (DECIMAL):** Kilómetros estimados del recorrido.
- **`scheduled_for` (TIMESTAMP):** Fecha y hora exacta a la que comenzará el evento. Solo se permiten fechas futuras en el momento de la inserción mediante restricciones `CHECK`.
- **`max_participants` (INTEGER):** Límite máximo de ciclistas permitidos. Si es nulo, la ruta es de aforo ilimitado.

5.2.3. Entidad `route_participants` (Tabla Intermedia N:M)

Debido a que un usuario puede apuntarse a infinitas rutas, y una ruta puede tener infinitos usuarios, es imperativo crear una entidad asociativa para romper la relación de Muchos a Muchos.

- **`route_id` (UUID - Clave Foránea):** Apunta a `routes.id`. Forma parte de la clave primaria compuesta.

- **`user_id` (UUID - Clave Foránea):** Apunta a `profiles.id`. Forma parte de la clave primaria compuesta para evitar que un mismo usuario se inscriba dos veces en el mismo evento.
- **`status` (VARCHAR 20):** Estado de la inscripción ('Confirmado', 'Cancelado').
- **`joined_at` (TIMESTAMP):** Registro del momento exacto en el que el ciclista reservó su plaza.

5.2.4. Entidad `user_reviews` (Sistema de Valoraciones Sociales)

El motor de confianza de la aplicación en la región de Extremadura.

- **`id` (UUID - Clave Primaria):** Identificador único del comentario.
- **`reviewer_id` (UUID - Clave Foránea):** Usuario que escribe la opinión.
- **`reviewed_id` (UUID - Clave Foránea):** Usuario que recibe la opinión. Restricción imperativa: `reviewer_id` no puede ser igual a `reviewed_id` (un usuario no puede autovalorarse).
- **`route_id` (UUID - Clave Foránea):** Ruta en la que ambos usuarios coincidieron. Esta relación es vital para evitar "spam"; solo puedes valorar a alguien si ambos estabais confirmados en la misma `route_id`.

- `punctuality_rating` **(INTEGER):** Puntuación del 1 al 5 sobre la puntualidad.
- `attitude_rating` **(INTEGER):** Puntuación del 1 al 5 sobre el compañerismo.
- `comment_body` **(TEXT):** Opinión libre escrita por el revisor.

5.3. Restricciones de Integridad y Acciones en Cascada

El modelado de datos no finaliza con la creación de tablas. Para mantener la coherencia del sistema ante acciones destructivas, se han programado reglas a nivel de motor SQL.

Por ejemplo, se ha configurado la regla `ON DELETE CASCADE` en las claves foráneas de los participantes. Esto significa que, si por algún motivo un administrador elimina una Ruta del sistema por contener información inapropiada, automáticamente se eliminarán todos los registros de inscripción en la tabla `route_participants` asociados a esa ruta. De igual manera, si un usuario solicita la baja de la plataforma ejerciendo su derecho al olvido, sus valoraciones previas se

anonimizarán aplicando un `ON DELETE SET NULL` al campo `reviewer_id`, preservando así la media aritmética del usuario que recibió la valoración, pero desvinculando por completo los datos personales del emisor.

6. DIARIO DE DESARROLLO (CRONOLOGÍA DETALLADA)

Este capítulo constituye el núcleo central de la memoria técnica. En él se documenta, de forma exhaustiva y cronológica, la evolución del sistema BicisExt desde su concepción teórica hasta su despliegue en producción. Cada entrada refleja no solo las tareas ejecutadas, sino también los bloqueos técnicos experimentados, las decisiones de arquitectura tomadas y las justificaciones metodológicas que respaldan dichas decisiones.

6.1. Primer Trimestre: Fase de Concepción y Diseño Estructural

12 de Septiembre de 2025: Presentación de la Asignatura y Validación de la Idea Teórica

El primer día de desarrollo oficial comenzó con la presentación de la directrices del Proyecto Final. Tras analizar las rúbricas de evaluación, dediqué la jornada íntegra a la fase de ideación y validación de mercado. El objetivo no era simplemente "hacer una web", sino resolver un problema arquitectónico y social real en mi entorno inmediato: la Comunidad Autónoma de Extremadura.

Comencé realizando un análisis DAFO (Debilidades, Amenazas, Fortalezas y Oportunidades) del panorama digital actual enfocado al ciclismo. Analicé aplicaciones líderes en el mercado como Strava, Wikiloc y Komoot. Durante esta ingeniería inversa a nivel de producto, detecté un patrón claro: todas estas plataformas son excelentes para la monitorización del rendimiento individual (tracking GPS, cálculo de vatios, desnivel acumulado) y para el descubrimiento de rutas asíncronas, pero fracasan estrepitosamente en la gestión comunitaria hiperlocal. Si un ciclista de Mérida quiere salir mañana a las 08:00 AM y

busca compañeros de su mismo nivel físico, Strava no ofrece una herramienta ágil para organizar ese encuentro en tiempo real.

Esta fragmentación obliga a los usuarios a depender de foros obsoletos o grupos de WhatsApp caóticos donde la información (hora, lugar, dificultad) se pierde entre cientos de mensajes. Aquí nació la justificación técnica de **BicisExt**.

Durante la tarde, realicé un estudio de viabilidad técnica (Feasibility Study). Me planteé si, como único desarrollador, era viable implementar un sistema de tracking GPS en tiempo real. La conclusión analítica fue negativa: procesar ráfagas de coordenadas geoespaciales por segundo saturaría la base de datos y requeriría una arquitectura de WebSockets excesivamente compleja para un Producto Mínimo Viable (MVP). Por tanto, pivoté la idea técnica: BicisExt no trackeará la ruta en vivo, sino que funcionará como un gestor de eventos asíncrono, enfocando la complejidad técnica en el sistema relacional de base de datos y en el algoritmo de confianza social (valoraciones post-ruta). Presenté este "Elevator Pitch" al tutor del proyecto, obteniendo la validación necesaria para avanzar.

19 de Septiembre de 2025: Creación de Imagen Corporativa y Diseño de Identidad Visual Adaptativa

Un error común en la ingeniería de software es menospreciar la interfaz hasta las fases finales. Sin embargo, en BicisExt, la Interfaz de Usuario (UI) está estrechamente ligada a las condiciones extremas de uso: el usuario visualizará la aplicación en exteriores, bajo la luz solar directa de Extremadura, probablemente con gafas de sol polarizadas y pulsaciones elevadas. Esto convierte el diseño corporativo en un problema de accesibilidad técnica, no solo de estética.

Dediqué esta jornada a establecer el "Design System" (Sistema de Diseño) fundacional de la aplicación. En primer lugar, abordé la teoría del color. Rechacé inmediatamente las interfaces oscuras ("Dark Mode" por defecto) porque el cristal de los smartphones refleja demasiada luz en exteriores, haciendo que los fondos negros actúen como un espejo. Establecí una paleta fundamentada en un fondo "Blanco Humo" (#F9FAFB) que maximiza el rebote de la retroiluminación de la pantalla, garantizando un contraste absoluto. Como color primario de acento, desarrollé un tono específico: "Verde Dehesa" (#3D5229). Este color no solo cumple con los estándares AAA de las normativas WCAG 2.1 sobre fondos claros, sino que psicológicamente ancla la aplicación a la identidad natural de los paisajes extremeños.

A continuación, procedí con la selección tipográfica. Descarté cualquier fuente Serif, ya que sus remates generan ruido visual en pantallas de alta densidad de píxeles (Retina/OLED) cuando el tamaño de la fuente

es inferior a 14px. Seleccioné la tipografía 'Inter'. Su diseño está matemáticamente optimizado para interfaces informáticas, poseyendo una altura de la 'x' (la proporción de las letras minúsculas respecto a las mayúsculas) inusualmente alta. Esto significa que un texto a 12px en BicisExt es tan legible como un texto a 14px en tipografías estándar como Arial o Helvetica, permitiéndome condensar más información de la ruta en la pantalla del móvil sin forzar la vista del ciclista.

Finalmente, trabajé en el diseño del logotipo. Utilizando software de gráficos vectoriales (SVG), diseñé una abstracción geométrica que fusiona el eslabón de una cadena de transmisión de bicicleta con un pin de geolocalización. La decisión de usar SVG puro y no imágenes PNG/JPG radica en el rendimiento web: un SVG es simplemente código matemático que el navegador renderiza al instante, pesando menos de 2 kilobytes y escalando infinitamente sin pérdida de resolución, lo cual es crítico para los tiempos de carga estipulados en los Requisitos No Funcionales.

26 de Septiembre de 2025: Elaboración del Contrato de Desarrollo y Recogida de Necesidades (Stakeholders)

Esta jornada marcó la transición del diseño abstracto a la especificación de ingeniería formal. Para simular un entorno profesional realista, dediqué el día a la elaboración de un SLA (Service Level Agreement) y al levantamiento de requisitos mediante la interacción con los "Stakeholders" (partes interesadas).

Realicé una serie de entrevistas de simulación con perfiles de ciclistas locales (representando a clubes de MTB de Cáceres y cicloturistas de carretera de la provincia de Badajoz). El objetivo de esta técnica de "Requirements Elicitation" era descubrir fricciones ocultas que yo, como desarrollador aislado, podría pasar por alto. El hallazgo más crítico de estas entrevistas fue el "Síndrome del Desajuste de Nivel". Los usuarios expresaron un temor profundo a unirse a una ruta pública y descubrir que el ritmo del grupo es profesional, dejándolos rezagados en mitad del campo, o por el contrario, que el grupo sea excesivamente lento, arruinando su entrenamiento.

Este descubrimiento me obligó a modificar la arquitectura lógica que tenía pensada. Comprendí que no bastaba con publicar una ruta; el sistema necesitaba un contrato de expectativas paramétrico.

Inmediatamente, redacté nuevos Requisitos Funcionales obligando a que la base de datos registre parámetros estrictos e inmutables para cada evento creado: 'Ritmo Medio Estimado (km/h)', 'Desnivel Positivo Acumulado (metros)', y 'Clasificación Técnica del Terreno'.

Además, se detectó el problema del absentismo (usuarios que reservan plaza y no se presentan). Para mitigar esto, definí a nivel teórico el "Algoritmo de Fiabilidad BicisExt". Consiste en que el backend calculará automáticamente un porcentaje de asistencia basado en el historial del usuario. Si un usuario tiene un 20% de asistencia, los organizadores de nuevas rutas podrán restringir su inscripción, creando un ecosistema autorregulado basado en la responsabilidad personal.

Finalmente, redacté el documento de alcances y límites. Dejé por escrito, de forma explícita, que la plataforma actuará exclusivamente como intermediario de información (Information Broker) y no como entidad aseguradora de los eventos. Esto es vital desde el punto de vista legal y de protección de datos (RGPD). También establecí los hitos de entrega, comprometiéndome a finalizar el modelo relacional de la base de datos para finales del mes de octubre.

03 de Octubre de 2025: Definición de Requisitos Funcionales y No Funcionales bajo el Estándar IEEE 830

Tras la recogida de información de los *stakeholders* locales realizada la semana anterior, esta jornada se destinó íntegramente a la Ingeniería de Requisitos. El objetivo era traducir el lenguaje humano, a menudo ambiguo y contradictorio, en especificaciones técnicas estrictas y medibles que el sistema informático de BicisExt pudiera procesar. Para dotar al proyecto de un rigor académico y profesional, me basé en las directrices del estándar IEEE 830 para la Especificación de Requisitos de Software (SRS).

Comencé redactando el catálogo de **Requisitos Funcionales (RF)**, que definen el comportamiento interno del sistema. El primer gran bloque fue el *Módulo de Autenticación y Perfilado (RF-01)*. Especificé que el registro no solo requeriría un correo electrónico, sino una verificación de doble paso (Magic Link o confirmación por email) para evitar la proliferación de cuentas bot o perfiles falsos que pudieran dinamitar la confianza de la comunidad extremeña. Además, definí el *Módulo de Gestión de Rutas (RF-02)* estableciendo reglas de negocio inmutables: un usuario no puede crear una ruta con fecha en el pasado, y es

obligatorio definir el "Tipo de Bicicleta" (Carretera, MTB, Gravel) para evitar accidentes por desinformación.

El requisito más complejo de definir fue el *Módulo de Confianza y Valoración (RF-03)*. Tuve que diseñar a nivel teórico la lógica condicional: el sistema solo permitirá a un "Usuario A" valorar a un "Usuario B" si, y solo si, la base de datos certifica que ambos estuvieron inscritos en el mismo "ID de Ruta" y la fecha de dicha ruta ya ha pasado ("Timestamp_Ruta < Timestamp_Actual"). Esto previene el "review bombing" (ataques de reseñas falsas).

Posteriormente, pasé a definir los **Requisitos No Funcionales (RNF)**, que dictan las restricciones y métricas de calidad del sistema. Aquí el contexto geográfico de Extremadura fue crucial. Sabiendo que gran parte de nuestro territorio tiene zonas de sombra en conectividad 4G/5G (especialmente en sierras y valles profundos), establecí un "Presupuesto de Rendimiento" (Performance Budget) estricto: *RNF-01 (Rendimiento)*: El *First Contentful Paint* (FCP) de la aplicación debe ser inferior a 1.2 segundos en redes 3G. Para cumplir esto, supe que tendría que implementar técnicas de carga diferida (Lazy Loading) en las imágenes y mapas.

Por último, redacté el *RNF-02 (Seguridad)*. Exigí que la aplicación debe ser invulnerable a inyecciones SQL y ataques XSS (Cross-Site

Scripting). Esta especificación me llevó a confirmar que el uso de Supabase (que parametriza las consultas por defecto) y React (que escapa el HTML inyectado en el DOM) cubrirían este requisito arquitectónico desde la base.

10 de Octubre de 2025: Desarrollo de Interfaces Gráficas y Prototipado UI/UX de Alta Fidelidad

Una vez que las reglas del sistema estaban escritas sobre papel, dediqué esta jornada a la construcción visual del producto. Me trasladé a Figma para materializar los requisitos en pantallas interactivas, adoptando una filosofía de diseño **"Mobile-First"**. Diseñar primero para la pantalla más pequeña te obliga a priorizar drásticamente la información; si una funcionalidad no cabe en un móvil, probablemente no sea esencial.

Inicié la sesión aplicando los principios de la Ley de Fitts, que dictamina que el tiempo necesario para alcanzar un objetivo es una función de la distancia y el tamaño del objetivo. En el contexto de un ciclista (que puede tener las manos frías, llevar guantes o estar en movimiento), los botones pequeños son un error crítico de usabilidad. Por ello, definí que todos los elementos interactivos (Call to Actions como "Unirme a la ruta", "Filtrar" o "Confirmar asistencia") tendrían un área táctil mínima de 48x48 píxeles independientes del dispositivo (dp).

Diseñé el flujo principal: el **Explorador de Rutas**. Decidí dividir la pantalla en dos capas. La capa base sería un mapa interactivo renderizado mediante una API cartográfica (como Mapbox o Google Maps), mostrando "pines" personalizados en las coordenadas de inicio de las rutas en Extremadura. La capa superior sería un panel deslizante (Bottom Sheet) que muestra las tarjetas de información (*Route Cards*). En estas tarjetas, apliqué una jerarquía visual estricta: lo más grande es la fecha y la hora, seguido del nivel de dificultad (con un código de colores semántico: verde para fácil, ámbar para medio, rojo para difícil) y finalmente la distancia en kilómetros.

Durante la tarde, me enfoqué en el **Dashboard Personal**. Diseñé la vista del perfil de usuario donde se exhibe el "Historial de Rutas" y el "Radar de Fiabilidad". En lugar de mostrar un simple número del 1 al 5, diseñé un gráfico de telaraña (Radar Chart) que evalúa al ciclista en tres ejes: Puntualidad, Compañerismo y Nivel Técnico real frente al declarado. Esto transforma datos fríos en una visualización muy rápida y fácil de interpretar para cualquier usuario que esté decidiendo si aceptar a un desconocido en su grupo de salida. Al finalizar el día, enlacé todas las pantallas en Figma, creando un prototipo navegable que simulaba una aplicación real.

17 de Octubre de 2025: Desarrollo de la Estructura de la Base de Datos y Tipado Geográfico

Con la interfaz diseñada y los requisitos aprobados, el sistema necesitaba un "cerebro" donde almacenar la información. Esta jornada fue de pura ingeniería de datos. Pasé del modelado conceptual al modelado lógico, diseñando la estructura interna que viviría en nuestro clúster de PostgreSQL alojado en Supabase.

La primera gran decisión arquitectónica del día fue la elección del tipo de dato para las **Claves Primarias (Primary Keys)**. En tutoriales básicos se suelen usar identificadores numéricos autoincrementales (ej. Usuario 1, Ruta 2, Ruta 3). Sin embargo, para BicisExt, esto suponía una vulnerabilidad grave conocida como IDOR (Insecure Direct Object Reference). Si la URL de una ruta es `bicisext.com/ruta/5`, un atacante malintencionado podría crear un bot que iterara `ruta/6`, `ruta/7` para raspar (scrape) todos los datos de los usuarios extremeños. Para evitarlo, definí que absolutamente todas las tablas utilizarían identificadores **UUID v4** (Universally Unique Identifier). Un UUID es una cadena alfanumérica de 36 caracteres generada criptográficamente, haciendo matemáticamente imposible predecir el siguiente registro.

A continuación, estructuré la tabla central: `rutas`. Aquí me encontré con un desafío interesante: ¿cómo guardo la ubicación del punto de encuentro? Podría haber creado dos columnas simples, `latitud` y `longitud`, con tipos de dato `FLOAT`. Sin embargo, pensando a futuro, si BicisExt crece y queremos hacer consultas complejas (como "muéstrame todas las rutas en un radio de 20km alrededor de Plasencia"), usar matemáticas básicas de trigonometría sobre coordenadas es extremadamente ineficiente a nivel de CPU. Por lo tanto, decidí investigar e implementar la extensión **PostGIS**, estándar en la industria para bases de datos geoespaciales. Definí la columna `punto_inicio` con el tipo de dato `GEOGRAPHY(Point, 4326)`, lo que permite a la base de datos realizar cálculos de distancias teniendo en cuenta la curvatura real de la Tierra.

Terminé la jornada diseñando las tablas de soporte: la tabla `usuarios` (vinculada al sistema de identidad mediante un trigger automático que crea un perfil público cada vez que alguien se registra) y la tabla crítica de `participantes`. Esta última es una tabla intermedia que rompe la relación de "Muchos a Muchos" entre usuarios y rutas, y a la que le añadí campos de auditoría como `fecha_inscripcion` y `estado` (confirmado, en lista de espera, o cancelado), vitales para el funcionamiento del ecosistema.

24 de Octubre de 2025: Definición del Modelo Relacional y Normalización de la Base de Datos

Con la estructura básica de las tablas definida en la sesión anterior, el objetivo arquitectónico de hoy fue aplicar las reglas de normalización de bases de datos para garantizar la integridad referencial y evitar anomalías de inserción, actualización o borrado. Un sistema como BicisExt, donde las rutas, los usuarios y las valoraciones están interconectados en tiempo real, no puede permitirse redundancias de datos que desincronicen la aplicación.

Comencé aplicando la Primera Forma Normal (1FN), asegurándome de que todos los atributos fueran atómicos. Por ejemplo, en las primeras iteraciones teóricas, consideré guardar las "etiquetas" de una ruta (ej. "Montaña", "Dura", "Paisajística") en una sola columna de texto separada por comas. Sin embargo, esto violaba la 1FN y habría hecho imposible realizar búsquedas eficientes (ej. `SELECT * FROM rutas WHERE etiqueta LIKE '%Montaña%'` es un antipatrón terrible para el rendimiento). Para solucionarlo, extraje estas características a campos fijos y booleanos o creé tablas de búsqueda auxiliares.

El mayor desafío técnico de la jornada fue consolidar la Tercera Forma Normal (3FN) en el sistema de valoraciones cruzadas. Inicialmente, pensé en añadir una columna `puntuacion_media` directamente en la tabla `usuarios`. Pero esto generaba una dependencia transitiva: la puntuación media no depende directamente de la clave primaria del usuario, sino que es un valor derivado matemáticamente de múltiples registros en la tabla `valoraciones`. Por tanto, eliminé esa columna de la tabla `usuarios` y programé una Vista Materializada (Materialized View) en PostgreSQL. Esta vista se encarga de calcular la media de reputación de un ciclista (sumando puntualidad y actitud) agrupando (GROUP BY) por su ID. De esta forma, el dato siempre es exacto y se calcula bajo demanda, manteniendo la base de datos inmaculada y libre de redundancias.

Finalmente, dediqué la tarde a establecer las Reglas de Integridad Referencial (Foreign Key Constraints). Diseñé la política de borrado en cascada (`ON DELETE CASCADE`) para la tabla intermedia de `participantes`. Si un usuario decide borrar su cuenta por motivos de privacidad (cumpliendo con la RGPD), el sistema automáticamente eliminará sus inscripciones futuras. Sin embargo, para las valoraciones que ese usuario ya haya emitido sobre otros ciclistas, apliqué una regla `ON DELETE SET NULL`. De este modo, el texto de la valoración y la

nota se mantienen en el sistema (para no perjudicar la nota media del ciclista evaluado), pero se desvincula permanentemente de la identidad del usuario que se ha dado de baja, garantizando el anonimato.

31 de Octubre de 2025: Presentación de Interfaces y Base de Datos al Cliente (Hito 1)

Esta jornada marcó el primer gran hito de entrega del proyecto. Simulando un entorno empresarial regido por metodologías ágiles, presenté el "Sprint Review" que englobaba todo el trabajo de septiembre y octubre ante el tutor del proyecto (que actuó bajo el rol de *Product Owner* o cliente final). El objetivo no era mostrar código, sino validar que la lógica de negocio y el diseño de interacción resolvían el problema inicial planteado para los ciclistas en Extremadura.

Comencé la presentación exponiendo el prototipo interactivo en Figma. Mostré el flujo completo: desde que un usuario extremeño se registra, hasta que visualiza el mapa interactivo, filtra una ruta en su provincia y pulsa el botón de "Unirme". La respuesta inicial fue muy positiva, destacando la limpieza de la interfaz ("Cubo Blanco") y el contraste de la paleta de colores verdes, que se alineaba perfectamente con la identidad visual solicitada.

Sin embargo, durante la fase de *feedback* y preguntas, el "cliente" detectó una brecha en la lógica de negocio que yo había pasado por alto. Se me sugirió que el tipo de terreno (asfalto, tierra, sendero, pista forestal) no podía ser simplemente un texto descriptivo opcional. El argumento fue irrefutable: en Extremadura, salir con una bicicleta de carretera de carbono por un camino de grava de la Vía de la Plata puede causar daños de miles de euros en la bicicleta o un accidente grave. Por tanto, el tipo de terreno debía ser un filtro de seguridad obligatorio desde la propia creación de la ruta.

Esta reunión demostró el inmenso valor de las metodologías ágiles frente al desarrollo tradicional en cascada (Waterfall). En lugar de descubrir este fallo en abril tras haber programado toda la aplicación, pude pivotar en el acto. Esa misma tarde, volví al esquema de la base de datos y modifiqué el modelo relacional en el acto para incluir el campo `terrain_type`. No lo declaré como un texto libre (`VARCHAR`), sino como un tipo `ENUM` restrictivo a nivel de motor SQL. Esto significa que la base de datos rechazará categóricamente cualquier inserción que no sea uno de los valores preaprobados ('Carretera', 'MTB', 'Gravel', 'Paseo'). Con esta modificación crítica implementada, di por concluida la fase de diseño y estructuración, firmando el documento de aprobación para pasar al desarrollo de código en el segundo trimestre.

07 de Noviembre de 2025: Elección y Justificación de Tecnologías (Stack Arquitectónico)

El inicio del segundo trimestre estuvo dedicado a una de las decisiones más críticas de cualquier proyecto de ingeniería de software: la elección del Stack Tecnológico. Elegir mal las herramientas en esta fase supone reescribir la aplicación meses después. Para BicisExt, necesitaba un ecosistema que permitiera construir una aplicación SPA (Single Page Application) altamente reactiva, rápida en móviles y con un backend robusto y seguro que pudiera gestionar sin que yo tuviera que dedicar semanas a configurar servidores Linux en la nube.

Tras comparar varias opciones, me he decantado por el Stack moderno: React + Vite + Tailwind CSS + Supabase. A continuación, detallo la justificación técnica de cada componente:

Para la capa de presentación (Frontend), evalué Angular, Vue.js y **React**. Me decanté por React (en su versión 18+) debido a su algoritmo de reconciliación basado en el *Virtual DOM*. En BicisExt, cuando un usuario está en la vista de "Detalle de Ruta" y alguien se apunta, el contador de "Plazas Disponibles" debe actualizarse al instante. Angular re-renderiza componentes pesados de forma bidireccional, lo que puede consumir mucha batería en móviles de gama baja. React, por el

contrario, detecta exactamente qué nodo del árbol HTML ha cambiado (el número de plazas) y actualiza solo ese fragmento, manteniendo un rendimiento estelar de 60 FPS en la interfaz.

Para el empaquetador del código, descarté el clásico Webpack a favor de **Vite**. Vite utiliza módulos de JavaScript nativos (ESM) en el navegador. Esto significa que mi entorno de desarrollo local arranca en menos de 300 milisegundos, frente a los 15-20 segundos que tardaba Create-React-App. En un proyecto de cientos de horas, este ahorro de tiempo de compilación es exponencial.

En el apartado de estilos, rechacé el uso de CSS tradicional o preprocesadores como SASS. En su lugar, adopté **Tailwind CSS**. Su paradigma de "Utility-First" me permite no tener que inventar nombres semánticos para clases (ej. `.card-ruta-verde-borde`). Simplemente aplico clases atómicas directamente en el JSX de React (ej. `bg-green-800 p-4 rounded-lg`). Esto asegura que el archivo CSS final generado en producción nunca supere los 10 o 15 kilobytes, cumpliendo estrictamente con el Requisito No Funcional de carga ultrarrápida en zonas de Extremadura con mala cobertura.

Finalmente, para la infraestructura y bases de datos, analicé AWS, Firebase y **Supabase**. Elegí Supabase por un motivo fundamental: es un Backend-as-a-Service (BaaS) construido sobre PostgreSQL puro. A

diferencia de Firebase (que es una base de datos NoSQL donde las relaciones complejas entre usuarios y valoraciones son una pesadilla técnica), Supabase me ofrece toda la potencia relacional que diseñé en octubre, integrando de forma nativa la autenticación de usuarios mediante JWT (JSON Web Tokens) y un sistema de almacenamiento de archivos (Buckets) para los avatares de los ciclistas. Esta arquitectura Serverless me permite actuar como un equipo de desarrollo completo (Full-Stack) siendo una sola persona.

14 de Noviembre de 2025: Estructuración Inicial de Documentación y Arquitectura Base del Código

Con el stack tecnológico (React, Vite, Tailwind, Supabase) formalmente aprobado, el objetivo de esta jornada fue levantar los cimientos del proyecto a nivel de software. En el desarrollo de aplicaciones web modernas, empezar a escribir código sin una arquitectura de carpetas predefinida conduce irremediablemente al "Código Espagueti" (Spaghetti Code), un antipatrón de diseño que hace que la aplicación sea imposible de mantener a largo plazo.

Inicié la sesión inicializando el repositorio local con Git y ejecutando el *scaffolding* (andamiaje) del proyecto mediante el comando `npm create vite@latest bicisext -- --template react`. Una vez

instaladas las dependencias base del entorno de Node.js, procedí a diseñar el árbol de directorios aplicando el principio de "Separación de Preocupaciones" (Separation of Concerns).

Definí la siguiente jerarquía estricta dentro del directorio `src`:

- `/assets`: Para almacenar los archivos estáticos inmutables (el logotipo vectorial en SVG diseñado en septiembre, tipografías locales y gráficos estructurales).
- `/components`: El núcleo del Diseño Atómico. Aquí creé subcarpetas para `atoms` (botones genéricos, *inputs* de texto), `molecules` (la tarjeta resumen de una ruta ciclista, campos de formulario con etiquetas) y `organisms` (el menú de navegación lateral, el pie de página, el contenedor del mapa interactivo).
- `/pages`: Exclusivamente para los componentes que actúan como vistas completas (Dashboard, Login, Perfil Público, Buscador de Rutas).
- `/hooks`: Para extraer toda la lógica de negocio y evitar ensuciar los componentes visuales. Por ejemplo, creé el archivo `useAuth.js` que se encargará exclusivamente de hablar con Supabase para saber si el usuario de Extremadura ha iniciado sesión.

- `/lib`: Carpeta reservada para la inicialización de librerías de terceros. Aquí configuré el archivo `supabaseClient.js`, inyectando las variables de entorno de forma segura.

Un punto crítico que abordé durante la tarde fue la configuración del control de calidad del código estático. En un proyecto de fin de ciclo, el código no solo debe funcionar, debe ser elegante y cumplir con los estándares de la industria. Por ello, configuré **ESLint** con las reglas de Airbnb y **Prettier**. Esto garantiza que cada vez que guardo un archivo, el código se formatea automáticamente, unificando el uso de comillas simples, tabulaciones y punto y coma. Además, configuré Tailwind CSS generando el archivo `tailwind.config.js`, donde inyecté las variables de la paleta de colores corporativa ("Verde Dehesa" y "Gris Asfalto"), logrando que toda la aplicación dependa de un único archivo de configuración visual.

21 de Noviembre de 2025: Definición de Puntos de los Manuales de Usuario y Técnico

Aunque la aplicación aún estaba en su fase temprana de codificación, la ingeniería de software exige que la documentación crezca en paralelo al código. Dejar los manuales para el último día del proyecto es una mala práctica que resulta en manuales pobres y desconectados de la realidad

del sistema. Por ello, dediqué este día a esquematizar la estructura de los dos grandes manuales requeridos: el Técnico y el de Usuario.

Me enfrenté a un reto de "cambio de contexto" (Context Switching). Tenía que redactar índices para dos públicos objetivo radicalmente opuestos:

1. **El Perfil Técnico (Profesores, Evaluadores o futuros Desarrolladores):** Este lector no necesita saber qué botón pulsar; necesita saber *por qué* el botón funciona.
2. **El Perfil de Usuario Final (El Ciclista Extremeño):** Este lector no sabe qué es una base de datos ni le interesa; necesita saber cómo recuperar su contraseña o cómo buscar una ruta en el Valle del Jerte sin perderse en la interfaz.

Para el **Manual Técnico**, definí un índice que cubriría la topología de la red, la explicación detallada de las políticas RLS (Row Level Security) que impiden a un usuario malicioso borrar rutas ajenas, y el diccionario de datos. Decidí que un punto crucial de este manual sería la explicación del flujo de los JSON Web Tokens (JWT) de Supabase: cómo el token se almacena en el almacenamiento local del navegador (Local Storage) y cómo se envía en cada petición para validar la identidad del ciclista ante el servidor.

Para el **Manual de Usuario**, adopté un enfoque empático. Dividí el manual en "Casos de Uso Reales" en lugar de descripciones de menús. Por ejemplo, en lugar de un título frío como "Pantalla de Filtrado", el punto del manual se titularía "Cómo encontrar una ruta adaptada a mi nivel físico". También estructuré un apartado especial para el "Sistema de Confianza y Valoraciones", ya que es la funcionalidad diferenciadora de BicisExt. En este punto, planifiqué explicar mediante infografías conceptuales cómo un ciclista recibe una nota tras finalizar una ruta y cómo esa nota afecta a su "Radar de Reputación".

05 de Diciembre de 2025: Desarrollo Inicial de Manuales (Fase de Abstracción)

Tras haber definido los índices en noviembre, esta jornada supuso una inmersión profunda en la redacción de los contenidos teóricos, entrando en lo que denomino la "Fase de Abstracción". Abstraer significa alejarte del código fuente puro y explicar la filosofía operativa del software.

Comencé redactando el bloque de **Gestión de Cuentas y Privacidad** para el manual de usuario. Al tratarse de una aplicación que maneja geolocalizaciones en la comunidad de Extremadura, la privacidad es una barrera psicológica gigante. Redacté un apartado explicando de forma llana, sin tecnicismos legales densos, cómo BicisExt anonimiza la ubicación exacta. Expliqué que el sistema no rastrea en tiempo real

dónde está el ciclista durmiendo, sino que el ciclista empuja (hace "Push") de una coordenada estática únicamente en el momento de establecer el punto de encuentro de la ruta.

Por la tarde, me enfoqué en el manual técnico, específicamente en el apartado de **Seguridad de la Base de Datos**. Documenté exhaustivamente las políticas RLS (Row Level Security) de Supabase. Escribí la justificación técnica de la política `auth.uid() = creator_id`. En el manual, expliqué que esta línea de código de PostgreSQL actúa como un guardián a nivel de kernel de la base de datos: si una petición HTTP maliciosa intenta ejecutar un comando `UPDATE` sobre una ruta, PostgreSQL intercepta la petición, lee el token JWT del usuario, comprueba si su ID coincide con el ID del creador de la ruta y, si no coinciden, bloquea la transacción instantáneamente lanzando un error 403 (Forbidden), protegiendo la integridad del sistema sin importar los fallos que pueda haber en el código Frontend.

Finalmente, desarrollé la documentación teórica del **Módulo de Valoraciones**. Redacté las reglas matemáticas del algoritmo que calcula la reputación. Establecí por escrito que la primera valoración de un usuario tendría un peso del 100%, pero a medida que el ciclista acumula decenas de rutas, el sistema aplicaría una media ponderada móvil. De esta forma, si un ciclista excelente tiene un mal día y recibe

una valoración de 1 estrella, su nota global no se desplomará drásticamente, dotando al sistema de una justicia matemática que evitará la frustración de la comunidad ciclista en la plataforma.

12 de Diciembre de 2025: Análisis Arquitectónico y Opciones de Despliegue (Infraestructura Cloud)

Con gran parte del código Frontend ya estructurado y los manuales en proceso de redacción, esta jornada se enfocó en la arquitectura de sistemas. BicisExt no podía quedarse eternamente viviendo en `localhost:5173` dentro de mi ordenador; necesitaba una infraestructura de servidores robusta, segura y de alta disponibilidad para dar servicio a toda Extremadura.

Realicé un análisis comparativo profundo entre los principales proveedores de alojamiento en la nube (Cloud Hosting) especializados en el despliegue de Single Page Applications (SPAs): AWS (Amazon Web Services), Netlify y **Vercel**.

Descarté AWS EC2 (Elastic Compute Cloud) rápidamente. Aunque ofrece un control absoluto sobre el servidor Linux, configurarlo requiere instalar Nginx o Apache, gestionar certificados SSL manualmente mediante Let's Encrypt y programar *scripts* de despliegue. Esto introducía un punto único de fallo (Single Point of Failure) y un mantenimiento que excedía el alcance de mis recursos como desarrollador en solitario.

La decisión final fue **Vercel**. La justificación técnica radica en su arquitectura de "Edge Network" (Red de Entrega de Contenido o CDN perimetral). Cuando subo BicisExt a Vercel, el código no se aloja en un único servidor en Madrid o París. Vercel distribuye los archivos estáticos (el HTML, los paquetes de JavaScript de React y el CSS de Tailwind) a través de decenas de nodos por todo el mundo. Esto es vital para el *Requisito No Funcional de Rendimiento* que establecí en octubre: si un ciclista en Las Hurdes abre la app, su petición HTTP no viaja hasta un servidor centralizado lejano, sino que el nodo CDN más cercano (probablemente en Lisboa o Madrid) le sirve los archivos en milisegundos.

Durante la tarde, documenté y configuré la arquitectura de las **Variables de Entorno** (`.env`). En el código de React, necesito conectar con la base de datos de Supabase usando una URL y una clave pública (Anon Key). Sin embargo, subir estas claves directamente al código fuente en

GitHub es una vulnerabilidad crítica de Nivel 1. Establecí un sistema dual: un archivo `.env.local` en mi máquina (excluido del repositorio mediante el archivo `.gitignore`) para hacer pruebas contra una base de datos de desarrollo, y preparé la documentación para inyectar las claves de producción directamente en el panel encriptado de Vercel.

19 de Diciembre de 2025: Pruebas de Despliegue en Entorno Local y Emulación de Producción

Último día de trabajo antes del receso de Navidad. Históricamente, en la ingeniería de software, desplegar código un viernes o antes de unas vacaciones es una práctica prohibida (conocida coloquialmente como "Read-Only Friday"). Por lo tanto, dediqué esta jornada a emular el entorno de producción de forma estrictamente local para garantizar que la transición en enero fuera limpia.

Hasta este momento, había estado desarrollando BicisExt utilizando el servidor de desarrollo de Vite (`npm run dev`). Este servidor es

maravilloso porque utiliza *Hot Module Replacement* (HMR), actualizando la pantalla en milisegundos cada vez que guardo un archivo. Sin embargo, este entorno no refleja la realidad. En desarrollo, Vite sirve los archivos como módulos ES nativos sin comprimir.

Para emular la producción, ejecuté el comando de compilación: `npm run build`. Aquí, la herramienta Rollup (el motor interno de Vite) tomó mis miles de líneas de código React y las "minificó", eliminando espacios, acortando nombres de variables y descartando el código CSS que no se estaba utilizando (Tree Shaking). El resultado fue una carpeta `/dist` increíblemente optimizada.

A continuación, levanté un servidor estático local para probar esta versión compilada (`npm run preview`). Y aquí me encontré con un clásico "Edge Case" (caso límite) de las SPAs. Al navegar por la aplicación haciendo clic en los botones, todo funcionaba perfecto. Pero si estando en la URL `http://localhost/ruta/detalle/123` pulsaba el botón de "Actualizar" (F5) en el navegador, el servidor devolvía un temido **Error 404 - Not Found**.

Análisis del fallo: Los servidores web tradicionales (como Apache o el estático que estaba usando) interpretan las URLs como rutas físicas de carpetas. Al pedir `/ruta/detalle/123`, el servidor buscaba

literalmente una carpeta llamada "ruta", dentro otra llamada "detalle" y un archivo "123.html". Al no existir, lanzaba un 404. *Solución implementada:* Tuve que programar una regla de reescritura de URLs (URL Rewriting). Configuré el servidor para que cualquier petición a cualquier ruta, independientemente de su nombre, devolviera siempre el archivo `index.html` principal. De esta forma, el archivo HTML se carga y es el enrutador de React (`react-router-dom`) el que lee la URL y decide qué componente dibujar en pantalla. Documenté este arreglo para asegurarme de que Vercel incluyera esta regla de reescritura en su archivo de configuración interno (`vercel.json`). Dejé el proyecto estabilizado y congelado (Code Freeze) para las fiestas.

09 de Enero de 2026: Pruebas de Despliegue en Vercel (Producción) e Integración Continua

Tras el periodo vacacional, el primer objetivo del año fue subir la plataforma a la nube, convirtiendo a BicisExt en un software accesible públicamente. Esta jornada estuvo marcada por la configuración de un pipeline de **Integración Continua y Despliegue Continuo (CI/CD)**.

En la ingeniería de software moderna, no se suben los archivos por FTP de forma manual. En su lugar, vinculé mi repositorio de GitHub

directamente con los servidores de Vercel. La arquitectura CI/CD funciona mediante *Webhooks*. Configuré el sistema de tal manera que cada vez que yo ejecuto un comando `git push origin main` (enviando nuevo código a la rama principal del repositorio), GitHub notifica a Vercel. Automáticamente, los servidores de Vercel clonan el código, instalan las dependencias (`npm install`), compilan el proyecto (`npm run build`) y, si no hay errores de sintaxis, sustituyen la versión antigua de la web por la nueva sin que el usuario final sufra ninguna caída del servicio (Zero Downtime Deployment).

El primer despliegue falló espectacularmente en tiempo de ejecución, a pesar de que la compilación había sido un éxito. Al abrir la URL pública generada por Vercel desde mi smartphone, me encontré con la infame **"Pantalla Blanca de la Muerte"** (White Screen of Death de React).

Proceso de depuración (Debugging): Conecté el móvil al ordenador y abrí las herramientas de desarrollador (DevTools) de Chrome para inspeccionar el dispositivo remoto. La consola del navegador mostraba un error crítico: `TypeError: Cannot read properties of undefined (reading 'auth')`. *Diagnóstico:* El cliente de Supabase no se estaba inicializando porque la URL y la clave de la base de datos eran nulas (`undefined`). Recordé el análisis que hice el 12 de diciembre. El código buscaba las variables

`import.meta.env.VITE_SUPABASE_URL`, pero como el archivo `.env.local` está ignorado por seguridad en Git, Vercel no tenía acceso a esas claves. *Resolución:* Entré en el panel de control de Vercel, navegué a *Project Settings* > *Environment Variables*, e introduje manualmente las claves criptográficas proporcionadas por Supabase. Volví a lanzar el proceso de despliegue y, en cuestión de 45 segundos, BicisExt apareció renderizado a la perfección en la pantalla.

Para culminar el día, realicé pruebas de campo. Salí de casa, desactivé el WiFi del teléfono y cargué la aplicación bajo una red 4G simulando las condiciones de un ciclista a pie de carretera. El mapa interactivo cargó los pines de las rutas de prueba de Extremadura en apenas 800 milisegundos, validando que el CDN de Vercel y el empaquetado de Vite estaban funcionando exactamente como lo había diseñado en la fase de requisitos.

16 de Enero de 2026: Fase de Pruebas Extensivas (QA) y Corrección de Errores Críticos

Con la aplicación web desplegada y accesible desde cualquier dispositivo, la jornada de hoy se centró íntegramente en el Aseguramiento de la Calidad (Quality Assurance - QA). En la ingeniería

de software, el desarrollador suele tener un sesgo cognitivo conocido como "Happy Path" (El Camino Feliz): probamos la aplicación haciendo exactamente lo que se supone que hay que hacer. Sin embargo, los usuarios reales hacen clics impredecibles, tienen conexiones inestables y cometen errores. Mi objetivo hoy era actuar como un usuario hostil e intentar "romper" BicisExt.

Comencé con pruebas de concurrencia y condiciones de carrera (Race Conditions). Simulé un escenario donde una ruta muy popular en la Sierra de Gata tenía una sola plaza disponible y dos usuarios intentaban apuntarse exactamente en el mismo milisegundo. *Análisis del fallo:* Al hacer clic compulsivamente (múltiples veces por segundo) en el botón "Unirme a la ruta", descubrí que el Frontend enviaba cuatro peticiones HTTP POST asíncronas casi simultáneas al servidor. Como la latencia de la red retrasaba la respuesta, el sistema registraba al mismo usuario cuatro veces en la tabla intermedia `route_participants`, ocupando plazas fantasma y corrompiendo la lógica del aforo máximo. *Resolución Técnica (Frontend):* Implementé un patrón de estado de interfaz (UI State Machine). Al detectar el primer clic, el botón cambia su estado interno a `isSubmitting = true`, deshabilitando el atributo HTML `disabled` y mostrando un icono de carga circular (Spinner). Hasta que el servidor no devuelve un código HTTP 200 (Éxito) o HTTP 400 (Error), el botón permanece bloqueado, impidiendo el doble clic accidental.

Resolución Técnica (Backend): Nunca se debe confiar exclusivamente en la validación del Frontend. Modifiqué el esquema físico de la base de datos PostgreSQL añadiendo una restricción de unicidad compuesta (Composite Unique Constraint). Ejecuté el comando `ALTER TABLE route_participants ADD CONSTRAINT unique_user_route UNIQUE (route_id, user_id);`. Con esta barrera a nivel de kernel de base de datos, si el Frontend llegara a fallar y enviara dos peticiones, la base de datos abortará la segunda transacción lanzando un error de violación de restricción, garantizando una integridad absoluta de los datos.

Por la tarde, realicé pruebas de regresión (Regression Testing) sobre el sistema de valoraciones. Detecté un fallo de seguridad lógico: un usuario avispado podía cambiar el ID en la URL para acceder al formulario de valoración de una ruta a la que nunca había asistido. Corregí esto implementando un Middleware de validación en el enrutador de React (`react-router-dom`) que, antes de renderizar la pantalla de valoración, hace una consulta rápida a Supabase para certificar que el `user_id` local tiene un estado de "Asistió" en la `route_id` correspondiente.

23 de Enero de 2026: Revisión Final y Auditoría de Documentación (Limpieza de Deuda Técnica)

Tras resolver los errores críticos reportados en la fase de QA, el proyecto entró en la fase de congelación de características (Feature Freeze). Esto significa que queda estrictamente prohibido añadir nuevas funcionalidades (por muy buenas ideas que parezcan) para evitar introducir nuevos *bugs* de última hora. La jornada de hoy se dedicó a saldar la "Deuda Técnica" acumulada durante los meses de desarrollo rápido.

El primer paso fue aplicar la regla del Boy Scout ("Deja el campamento más limpio de lo que lo encontraste"). Hice un barrido completo por todo el código fuente de la carpeta `src/`.

- **Eliminación de código muerto:** Borré todos los `console.log()`, `console.error()` y código comentado que había utilizado para depurar en noviembre y diciembre. En producción, dejar instrucciones de consola no solo ensucia el código, sino que puede revelar estructuras de datos internas a usuarios que abran las DevTools del navegador.
- **Refactorización de Componentes:** Detecté que el componente `RouteCard.jsx` había crecido hasta las 400 líneas de código

porque gestionaba tanto la interfaz (UI) como la lógica de formateo de fechas y colores de dificultad. Extraje esa lógica a funciones de ayuda puras (Helper functions) dentro de una nueva carpeta `src/utils/formatters.js`, reduciendo el componente principal a unas elegantes 80 líneas mucho más legibles.

- **Documentación en el Código:** Escribí comentarios formales utilizando el estándar JSDoc en los *Hooks* personalizados más complejos (como `useRouteFilters`). Esto asegura que si el tribunal evaluador revisa mi código en GitHub, entenderá al instante qué parámetros recibe cada función y qué tipo de datos devuelve.

La segunda mitad del día se enfocó en el repositorio público. Un proyecto de software no está terminado si su archivo `README.md` es pobre. Redacté un archivo Léame profesional, detallando los prerrequisitos de instalación (versión de Node.js necesaria), los pasos exactos para clonar el repositorio, y un glosario con los comandos de inicialización (`npm run dev`, `npm run build`). Además, creé un archivo `.env.example` subido al repositorio con claves en blanco. De esta forma, cualquier evaluador o desarrollador que clone BicisExt sabrá exactamente qué variables de entorno necesita pedir a Supabase para levantar el proyecto en su máquina local en menos de cinco minutos.

30 de Enero de 2026: Preparación para la Presentación, Defensa y Población de Datos

Un proyecto técnicamente perfecto puede ser suspendido si no se sabe comunicar su valor. Con el código auditado y subido a Vercel, esta jornada supuso un cambio de mentalidad: dejé de ser un desarrollador para convertirme en un presentador de producto. El objetivo era preparar la defensa ante el tribunal académico.

El primer obstáculo para una presentación exitosa es el "Síndrome de la App Vacía". Si me presento ante el tribunal, abro BicisExt y el mapa de Extremadura está en blanco porque no hay usuarios reales usándola todavía, la percepción psicológica será de un proyecto inacabado o pobre. La aplicación necesitaba "semillas" (Seed Data).

- **Creación del Script de Seeding:** En lugar de crear usuarios y rutas a mano (lo cual me llevaría días), programé un pequeño *script* en Node.js utilizando la librería `Faker.js`.
- **Población Estratégica:** Este script generó automáticamente 50 perfiles de usuarios ficticios (con avatares genéricos y biografías creíbles) y los insertó en la base de datos de Supabase. A continuación, el script generó 30 rutas ciclistas estratégicamente distribuidas por la geografía extremeña: 10 rutas de carretera por

el Valle del Jerte, 10 de MTB extremo por Las Hurdes y 10 rutas de gravel alrededor del embalse de Proserpina en Mérida. Además, el script simuló inscripciones y cruzó valoraciones entre ellos para que el "Radar de Reputación" de los perfiles mostrara gráficos reales y coloridos.

Una vez que la aplicación cobró vida con estos datos, dediqué la tarde a estructurar el hilo conductor (Storytelling) de la presentación. Decidí que no iba a aburrir al tribunal mostrando líneas de código en una diapositiva. La estructura de la defensa se dividiría en cuatro actos:

1. **El Problema:** La desconexión de los ciclistas en zonas rurales y la falta de confianza al quedar con desconocidos por foros.
2. **La Solución:** BicisExt y su algoritmo de valoraciones.
3. **La Demo en Vivo:** Una navegación real por la web alojada en Vercel, demostrando la fluidez de React y los filtros instantáneos del mapa geolocalizado.
4. **La Arquitectura Bajo el Capó:** Una explicación de alto nivel del stack Serverless, justificando la elección de Supabase y la implementación de las normativas de seguridad.

Para mitigar el riesgo de que el día de la presentación fallara el WiFi del instituto (la pesadilla de cualquier desarrollador), configuré mi teléfono

móvil como punto de acceso (Tethering) de respaldo y grabé un vídeo navegando por la app a modo de plan C.

06 de Febrero de 2026: Implementación de Feedback y Accesibilidad Web (Normativas WCAG 2.1)

Tras la presentación preliminar y las pruebas con los primeros datos simulados, esta jornada se dedicó exclusivamente a refinar la Interfaz de Usuario (UI) centrándome en un pilar fundamental de la ingeniería web moderna: la Accesibilidad (a11y). Una aplicación deportiva orientada a la comunidad de Extremadura debe ser inclusiva; no todos los ciclistas tienen una visión perfecta de 20/20, especialmente cuando visualizan una pantalla bajo la intensa luz solar o si padecen daltonismo.

Mi objetivo técnico fue auditar y adaptar BicisExt para cumplir con el nivel AA de las normativas WCAG 2.1 (Web Content Accessibility Guidelines).

- **Auditoría de Contraste Cromático:** Utilicé herramientas de análisis de contraste y descubrí que algunas alertas secundarias

que utilizaban un tono gris claro sobre el fondo "Blanco Humo" tenían un ratio de contraste de 3.2:1, insuficiente para el estándar AA (que exige un mínimo de 4.5:1 para texto normal). Modifiqué las variables globales de Tailwind CSS, oscureciendo el gris (`text-gray-500` a `text-gray-700`) para garantizar la legibilidad sin sacrificar la estética minimalista del diseño.

- **Semántica HTML y Lectores de Pantalla:** Revisé minuciosamente el árbol DOM (Document Object Model) generado por React. En las primeras iteraciones, muchos botones eran simples etiquetas `<div>` con eventos `onClick`. Esto es un antipatrón terrible de accesibilidad porque los lectores de pantalla (como VoiceOver o NVDA) no los reconocen como elementos interactivos. Refactoricé estos componentes para utilizar etiquetas `<button>` nativas. Además, añadí atributos `aria-label` a todos los iconos de la aplicación (por ejemplo, el icono de la bicicleta de montaña ahora tiene el atributo `aria-label="Ruta de tipo Mountain Bike"`), permitiendo que los usuarios con discapacidad visual interactúen con el mapa.
- **Navegación por Teclado:** Un requisito estricto de accesibilidad es que la web pueda operarse íntegramente sin ratón. Configuré los estados `:focus-visible` en Tailwind para que, al navegar pulsando la tecla "Tabulador", un anillo de enfoque (Focus Ring)

del color "Verde Dehesa" envuelva claramente el elemento seleccionado, permitiendo una navegación espacial perfecta por el *Dashboard* de rutas.

13 de Febrero de 2026: Optimización de Base de Datos y Revisión de Ciberseguridad

Con la accesibilidad resuelta, dediqué el día de hoy a la "fontanería" interna del sistema: el rendimiento a gran escala y la seguridad de los datos. Hasta ahora, las consultas a PostgreSQL en Supabase eran rápidas porque la base de datos tenía pocos registros. Sin embargo, como ingeniero de software, debo diseñar para el éxito; ¿qué pasará cuando BicisExt tenga 50.000 rutas históricas en Extremadura?

El primer gran reto fue la **Optimización del Rendimiento (Query Tuning)**.

- Análisis de cuellos de botella: Detecté que la consulta que alimenta el *Dashboard* principal (buscar rutas futuras ordenadas

por fecha) realizaba un *Sequential Scan* (escaneo secuencial). Es decir, el motor leía fila por fila toda la tabla para encontrar los resultados.

- Solución técnica: Implementé **Índices B-Tree (B-Tree Indexes)**. Ejecuté comandos SQL directamente en la consola de Supabase: `CREATE INDEX idx_routes_date ON routes(scheduled_for);` y otro índice en la columna de ubicación `CREATE INDEX idx_routes_location ON routes USING GIST (meeting_point);`. Al aplicar estos índices matemáticos, el tiempo de búsqueda para rutas futuras bajó drásticamente de 150ms a menos de 12ms , garantizando que las búsquedas sigan siendo instantáneas aunque tengamos miles de rutas registradas.

La tarde se destinó íntegramente a la **Ciberseguridad**.

- Refuerzo de las políticas RLS (Row Level Security): Realicé una auditoría exhaustiva de los permisos. Comprobé empíricamente que, aunque un atacante extrajera su token JWT (JSON Web Token) válido, abriera la terminal de su ordenador y lanzara peticiones `DELETE` manuales hacia la API REST de Supabase, la base de datos lo bloqueaba. La regla `auth.uid() = creator_id` funcionó como un muro de contención a nivel de

kernel, demostrando que es técnicamente imposible que un usuario borre o modifique una ruta que no ha creado él mismo.

- Prevención contra Enumeración de Usuarios: Configuré los *triggers* de registro para asegurar que los correos electrónicos no quedaran expuestos en la tabla pública de `perfiles`, bloqueando vectores de ataque de suplantación de identidad (Phishing).

20 de Febrero de 2026: Generación de Datos Simulados (Volumen) y Pruebas de Estrés (Load Testing)

Si bien el 30 de enero poblamos la base de datos con unos pocos registros (Seeding) para que la app no se viera vacía en la demostración, hoy el objetivo fue completamente diferente: someter a los servidores y a la base de datos a un castigo severo. Esta es una fase crítica conocida como "Pruebas de Estrés" (Stress Testing).

El objetivo de esta sesión era responder a la pregunta: ¿Se caerá el servidor de BicisExt si 500 ciclistas intentan apuntarse a la misma marcha cicloturista en Cáceres en el mismo minuto?

- **Configuración del Entorno de Pruebas:** Para no saturar mi propio ancho de banda, utilicé una herramienta profesional de carga en la nube llamada Apache JMeter (o herramientas modernas como K6). Escribí un *script* de automatización que simulaba el comportamiento de usuarios humanos.
- **Ejecución del Ataque Simulado:** Lancé una avalancha de 1.000 peticiones HTTP concurrentes contra el *endpoint* de inscripción a rutas de la API de Supabase. El script intentaba inscribir a cientos de usuarios ficticios en una ruta que solo tenía un límite de 50 plazas.
- **Análisis de Resultados y Concurrencia:** Los resultados fueron espectaculares y validaron el diseño de la arquitectura. Supabase gestionó el *Connection Pooling* (grupo de conexiones) sin colapsar. De las 1.000 peticiones, las primeras 50 devolvieron un código HTTP 201 (Creado, plaza reservada). La petición número 51 y todas las siguientes devolvieron correctamente un código HTTP 409 (Conflicto), ya que el límite de participantes (`max_participants`) se había alcanzado.

- Conclusión de la prueba: La integridad referencial y las transacciones ACID de PostgreSQL mantuvieron una coherencia absoluta. No se produjo ningún error de concurrencia ("Race Condition") y el servidor no sufrió caídas, demostrando que BicisExt es una aplicación robusta, elástica y completamente preparada para un lanzamiento a producción masivo

27 de Febrero de 2026: Cierre de Documentación y Manual de Despliegue

Tras haber superado con éxito las pruebas de estrés (Load Testing) de la semana anterior, el sistema había demostrado su solvencia técnica. Sin embargo, en la ingeniería de software profesional, un producto no está verdaderamente terminado si su infraestructura no es replicable.

Esta jornada se dedicó íntegramente a redactar la documentación técnica final, con especial foco en el Manual de Despliegue.

El objetivo principal de este manual era garantizar que cualquier otro desarrollador (o el tribunal evaluador) pudiera tomar el código fuente de mi repositorio y levantar un clon exacto de BicisExt en menos de 30 minutos. Para ello, documenté exhaustivamente el proceso de configuración del *Backend as a Service* (BaaS). Escribí los *scripts* de SQL exactos que deben ejecutarse en la consola de Supabase para recrear las tablas de perfiles, rutas y participantes , así como las funciones y *triggers* automatizados que gestionan el cálculo de la reputación social.

Por la tarde, me enfoqué en documentar el *pipeline* de Integración y Despliegue Continuo (CI/CD) alojado en Vercel. Redacté un apartado crítico sobre la gestión de las Variables de Entorno (.env). Dejé constancia estricta de que las claves de la API pública y la URL del proyecto en Supabase jamás deben subirse al repositorio de GitHub por motivos de ciberseguridad, detallando el proceso de inyección segura de estas credenciales directamente en el panel de configuración de Vercel.

05 de Marzo de 2026: Congelación de Código (Code Freeze) y Versionado Semántico

A exactamente dos semanas de la entrega final, declaré oficialmente el estado de *Hard Code Freeze* (Congelación Estricta de Código). Aunque durante el desarrollo detecté pequeñas oportunidades de mejora visual

y funcionalidades secundarias (como añadir un chat interno), introducir código nuevo a estas alturas del proyecto violaría los principios de estabilidad, arriesgándome a inyectar *bugs* críticos de regresión. La prioridad absoluta pasó a ser la estabilidad del sistema.

Con el código base bloqueado para nuevas funcionalidades, procedí a implementar un sistema de Versionado Semántico (SemVer). A lo largo de los meses anteriores, el desarrollo se había mantenido en versiones preliminares (v0.x.x). Hoy, tras una última pasada de revisión de las normativas de accesibilidad WCAG 2.1 y confirmar el rendimiento del empaquetador Vite, generé el *release* oficial en GitHub bajo la etiqueta **v1.0.0**.

Este hito marcó el paso de BicisExt de un prototipo de estudiante a un Producto Mínimo Viable (MVP) listo para producción. Adjunté a esta versión un archivo *Release Notes* (Notas de la Versión) detallando las capacidades de la plataforma: gestión asíncrona de eventos hiperlocales, filtrado geográfico mediante PostGIS y el motor de valoración cruzada que soluciona el vacío digital del cicloturismo en Extremadura.

12 de Marzo de 2026: Preparación de la Defensa y Creación de Material Audiovisual

Un proyecto con una arquitectura impecable puede fracasar si no se sabe comunicar su valor. Con el producto ya en su versión 1.0.0, cambié mi rol de desarrollador *Full-Stack* al de *Product Owner*. El objetivo de esta jornada fue diseñar el *Storytelling* de la defensa ante el tribunal académico y blindarme ante posibles fallos técnicos el día de la presentación.

Estructuré la exposición en torno a tres ejes fundamentales:

1. **El Problema Social:** La dependencia de grupos de mensajería caóticos y el "Síndrome del Desajuste de Nivel" que sufren los ciclistas extremeños.
2. **La Solución Lógica:** La demostración de cómo el Algoritmo de Fiabilidad mitiga el absentismo y construye un entorno de confianza.
3. **El Músculo Técnico:** La justificación del *Stack* elegido (React + Supabase) y la robustez del modelo relacional en PostgreSQL frente a bases de datos NoSQL.

Para el componente visual, preparé un entorno de demostración poblando la base de datos con rutas ficticias en ubicaciones clave como la Vía de la Plata y el Valle del Jerte. Además, consciente de la Ley de Murphy, apliqué una estrategia de mitigación de riesgos: grabé un *screencast* (vídeo de captura de pantalla) de 5 minutos navegando por la aplicación, filtrando rutas y simulando un registro completo. Este material audiovisual serviría como mi "Plan B" en caso de que la red Wi-Fi del centro educativo fallara el día de la defensa, asegurando que el tribunal pudiera ver la fluidez de las interacciones y la SPA sin depender de una conexión a internet en vivo.

19 de Marzo de 2026: Entrega Definitiva y Defensa del Proyecto

Día 19 de marzo de 2026 , se consolida todo el esfuerzo de los últimos seis meses en la entrega y defensa de la Memoria Técnica Integral del Proyecto BicisExt.

La mañana comenzó con una última auditoría visual, comprobando la generación correcta del Índice Interactivo y la alineación de todos los diagramas arquitectónicos y modelos de entidad-relación. A continuación, procedí a generar el archivo PDF definitivo y a realizar la entrega formal a través de la plataforma virtual del centro educativo.

Posteriormente, me enfrenté a la defensa pública ante el tribunal evaluador. La estrategia de mitigación de riesgos planificada la semana anterior dio sus frutos, permitiéndome exponer con la seguridad de tener un respaldo audiovisual, aunque afortunadamente el despliegue en vivo desde los nodos CDN de Vercel funcionó de manera impecable con una latencia mínima.

Este proyecto no solo culmina mi ciclo formativo en Desarrollo de Aplicaciones Web/Multiplataforma , sino que me ha permitido enfrentarme a los retos reales de la industria del *software*: desde la captura inicial de requisitos con *stakeholders* y el diseño de la interfaz de usuario bajo condiciones extremas de uso , hasta el modelado complejo de bases de datos relacionales y el despliegue de infraestructuras *Serverless*. BicisExt queda así documentado, entregado y listo para escalar.

7. MANUAL TÉCNICO PARA DESARROLLADORES

Este manual proporciona la documentación arquitectónica y las directrices operativas necesarias para comprender, mantener y escalar el ecosistema de BicisExt. Está dirigido a perfiles técnicos que necesiten auditar el código fuente, replicar el entorno de desarrollo o ampliar las funcionalidades del sistema.

7.1. Arquitectura Base y Ecosistema

El proyecto se sustenta sobre un paradigma "JAMstack" (JavaScript, APIs y Markup), evolucionado hacia una arquitectura "Serverless" y *Backend as a Service* (BaaS). Esta decisión técnica desacopla completamente la capa de presentación de la capa de persistencia de datos.

El *stack* tecnológico seleccionado es React + Vite + Tailwind CSS + Supabase:

- **Frontend:** React (v18+) gestiona la reactividad mediante su *Virtual DOM*. Vite actúa como entorno de construcción (Bundler) utilizando módulos ESM nativos para acelerar el arranque en desarrollo y Rollup para optimizar la compilación en producción.
- **Estilos:** Tailwind CSS mediante su enfoque *Utility-First*, procesado con PurgeCSS para eliminar el código no utilizado en producción.

- **Backend y Base de Datos:** Supabase proporciona una base de datos PostgreSQL relacional completa. Se aprovecha su integración nativa para la autenticación y el almacenamiento de archivos (Buckets).
-

7.2. Estructura de Directorios (Separation of Concerns)

Para evitar el antipatrón de diseño conocido como "Código Espagueti", el repositorio sigue un principio estricto de Separación de Preocupaciones. Todo el código fuente reside dentro del directorio src/:

- **/assets:** Almacena los archivos estáticos inmutables, como el logotipo vectorial en formato SVG y gráficos estructurales.
- **/components:** Núcleo visual basado en la metodología de Diseño Atómico. Contiene subdirectorios para *atoms* (botones genéricos, *inputs*), *molecules* (tarjetas resumen de rutas) y *organisms* (menú de navegación, contenedor del mapa).
- **/pages:** Reservado exclusivamente para componentes que actúan como vistas completas de la aplicación (Dashboard, Login, Buscador de Rutas).
- **/hooks:** Extrae la lógica de negocio para no ensuciar los componentes visuales. Por ejemplo, el archivo useAuth.js se encarga de la comunicación con Supabase para verificar las sesiones iniciadas.
- **/lib:** Carpeta destinada a la inicialización de librerías de terceros. Aquí se ubica supabaseClient.js, encargado de inyectar las variables de entorno de forma segura.

7.3. Autenticación y Flujo de JSON Web Tokens (JWT)

El sistema no almacena contraseñas directamente; delega esta responsabilidad en el servicio de identidad de Supabase. El flujo de seguridad se basa en el estándar de JSON Web Tokens (JWT).

Una vez que un usuario se autentica con éxito, Supabase emite un JWT firmado criptográficamente. Este *token* se almacena de forma segura en el almacenamiento local del navegador (*Local Storage*) del cliente. En cada petición posterior a la base de datos (por ejemplo, para inscribirse en una ruta), el cliente de Supabase inyecta automáticamente este *token* en las cabeceras HTTP para validar la identidad del ciclista ante el servidor.

7.4. Seguridad a Nivel de Fila (RLS) en PostgreSQL

La seguridad de BicisExt no depende exclusivamente de validaciones en el Frontend. Se ha implementado un esquema robusto de *Row Level Security* (RLS) directamente en el motor de PostgreSQL.

La política de seguridad más crítica se define bajo la instrucción auth.uid() = creator_id. Si una petición HTTP maliciosa intenta ejecutar

un comando UPDATE sobre una ruta ajena, PostgreSQL intercepta la transacción, lee el *token* JWT del usuario, y comprueba si coincide con el ID del creador de la ruta. Si la validación falla, el *kernel* de la base de datos bloquea la operación instantáneamente lanzando un error HTTP 403 (Forbidden), protegiendo la integridad del ecosistema independientemente de vulnerabilidades en la capa cliente.

7.5. Modelado Geoespacial y Prevención de Vulnerabilidades

El diseño de la base de datos contempla medidas preventivas contra ataques de extracción de datos masiva (*Scraping*). Para evitar la vulnerabilidad IDOR (*Insecure Direct Object Reference*), absolutamente todas las tablas utilizan identificadores UUID v4 (*Universally Unique Identifier*). Al ser cadenas alfanuméricas de 36 caracteres generadas criptográficamente, resulta matemáticamente imposible para un atacante predecir el siguiente registro alterando la URL.

Para la gestión de los puntos de encuentro de las rutas, se ha descartado el uso de columnas FLOAT para latitud y longitud. En su lugar, se ha implementado la extensión geoespacial PostGIS, definiendo la columna de ubicación con el tipo de dato GEOGRAPHY (Point, 4326). Esto permite realizar consultas espaciales complejas (búsqueda por radio de cercanía) delegando la carga matemática trigonométrica al servidor de base de datos de manera altamente eficiente.

8. MANUAL DE USUARIO Y GUÍA DE COMUNIDAD

Este manual está diseñado para acompañar al ciclista en su experiencia dentro del ecosistema BicisExt. A diferencia de la documentación técnica, aquí no hablaremos de bases de datos ni de código ; nos centraremos en cómo utilizar la plataforma para organizar rutas, encontrar compañeros y construir una comunidad ciclista segura en Extremadura.

8.1. Gestión de Cuentas y Privacidad Real

La seguridad es el pilar de BicisExt. Para garantizar que todos los perfiles corresponden a ciclistas reales y evitar cuentas falsas (bots) que dañen la confianza de la comunidad, el registro exige una verificación de doble paso mediante correo electrónico.

Tu privacidad geográfica está garantizada: Entendemos que compartir tu ubicación puede generar dudas. BicisExt **no** rastrea tu

ubicación en tiempo real mientras te desplazas ni guarda un historial de por dónde te mueves. El sistema de mapas únicamente utiliza una coordenada estática en el momento exacto en el que decides fijar el "Punto de Encuentro" para iniciar una ruta.

8.2. Cómo encontrar una ruta adaptada a tu nivel físico

El "Síndrome del Desajuste de Nivel" (apuntarte a una ruta y descubrir que el grupo va demasiado rápido o demasiado lento) es uno de los mayores miedos del cicloturista. Para evitarlo, hemos diseñado un buscador intuitivo:

- **Exploración por Mapa:** Al abrir la aplicación, el sistema te mostrará un mapa interactivo con pines que representan los puntos de inicio de las rutas cercanas a tu ubicación actual.
- **Lectura Rápida (Tarjetas de Ruta):** Al pulsar sobre un pin o deslizar el panel inferior, verás la tarjeta resumen de la ruta. Para facilitar la lectura en exteriores, hemos aplicado un código de colores semántico a la dificultad: **Verde** (Fácil), **Ámbar** (Medio) y **Rojo** (Difícil).

- **Datos Inmutables:** Antes de unirte, podrás revisar el ritmo medio estimado, el desnivel positivo acumulado y la distancia total, asegurando que la salida encaja con tu entrenamiento.

8.3. Cómo organizar tu propia salida grupal

Crear una ruta en BicisExt es un proceso guiado de solo 3 pasos, diseñado para hacerse rápidamente desde el teléfono móvil.

1. **Datos Básicos:** Define un título descriptivo (ej. "Subida al Castillo de Montánchez") y establece la fecha y hora de salida.
2. **Selección del Terreno (Obligatorio):** Por motivos de seguridad, es estrictamente obligatorio marcar el tipo de terreno (Carretera, MTB o Gravel). Esto evita confusiones peligrosas, como presentarse con una bicicleta de asfalto a una ruta por pistas de tierra.
3. **Punto de Encuentro:** Navega por el mapa interactivo y coloca el pin en el lugar exacto donde el grupo deberá reunirse. Si lo

deseas, puedes establecer un límite máximo de plazas para la ruta.

4.

8.4. Sistema de Confianza y Valoraciones (Radar de Fiabilidad)

BicisExt no es solo una app de mapas; es un motor de confianza social. El objetivo es que sepas exactamente con quién vas a rodar antes de salir de casa.

¿Cómo funcionan las valoraciones? Para evitar reseñas falsas o malintencionadas (*review bombing*), el sistema solo te permitirá valorar a otro ciclista si la base de datos certifica que ambos estuvisteis inscritos en la misma ruta y la fecha del evento ya ha pasado.

El Radar de Reputación: En tu perfil público y en el de los demás, no verás un simple número frío. Verás un gráfico visual en forma de telaraña (Radar Chart) que evalúa a cada ciclista en tres ejes fundamentales:

- **Puntualidad:** ¿Llega a la hora acordada al punto de encuentro?.
- **Compañerismo / Actitud:** ¿Es solidario si alguien pincha o se queda rezagado?.
- **Nivel Técnico:** ¿Su nivel físico real coincide con el que declara en su perfil?.

Justicia Matemática y Absentismo: Si eres un ciclista excelente pero un día tienes un contratiempo y recibes una mala valoración, no te preocupes. El algoritmo de BicisExt utiliza una media ponderada móvil: tu primera valoración vale el 100%, pero a medida que acumulas rutas,

el sistema suaviza los impactos aislados para que una sola mala nota no hunda tu reputación global.

Nota de Convivencia: El sistema también monitoriza el absentismo. Si un usuario se apunta sistemáticamente a rutas y no se presenta, su porcentaje de asistencia bajará, y los organizadores podrán restringir su participación en eventos futuros

9. PLAN DE PRUEBAS (QA) Y DESPLIEGUE EN PRODUCCIÓN

Para garantizar que BicisExt no solo funcionara en un entorno de desarrollo ideal ("localhost"), sino que soportara las condiciones reales y caóticas del uso público en Extremadura, se ejecutó un riguroso Plan de Pruebas y un proceso de Integración Continua (CI/CD). El objetivo de esta fase fue abandonar el sesgo del "Camino Feliz" (Happy Path) y actuar como un usuario hostil para detectar y corregir vulnerabilidades .

9.1. Aseguramiento de la Calidad (QA) y Casos Límite

Las pruebas funcionales se centraron en romper la lógica de negocio de la aplicación simulando comportamientos impredecibles y latencias de red.

- **Gestión de Concurrencia (Condiciones de Carrera):** Se simuló un escenario crítico donde una ruta tenía una sola plaza disponible y múltiples peticiones se enviaban en el mismo milisegundo . Se detectó que la latencia permitía inscribir al usuario varias veces en la tabla route_participants, ocupando plazas "fantasma" .
- **Doble Resolución (Frontend y Backend):** En la capa visual, se implementó una máquina de estados (UI State Machine) que bloquea el botón y muestra un indicador de carga (isSubmitting = true) hasta recibir respuesta del servidor . En la base de datos, se aplicó una restricción de unicidad compuesta (UNIQUE (route_id, user_id)) para que PostgreSQL aborte cualquier transacción duplicada a nivel de *kernel*, garantizando la integridad absoluta del aforo .
- **Pruebas de Regresión Lógica:** Se descubrió que un usuario podía manipular la URL para acceder al formulario de valoración de una ruta a la que no había asistido . Esto se mitigó programando un *Middleware* en React Router que valida contra Supabase si el usuario realmente tiene el estado de "Asistió" antes de renderizar la pantalla de puntuación .

9.2. Pruebas de Estrés y Rendimiento (Load Testing)

Para certificar la viabilidad de la plataforma ante eventos masivos o marchas cicloturistas populares, se sometió al servidor a pruebas de carga máxima .

- **Simulación de Avalancha:** Utilizando herramientas de automatización como Apache JMeter o K6, se inyectaron 1.000 peticiones HTTP concurrentes contra el *endpoint* de inscripción de una ruta limitada a 50 plazas .
- **Análisis de Respuesta:** Supabase gestionó el *Connection Pooling* (grupo de conexiones) de manera impecable y sin colapsar . Las primeras 50 peticiones devolvieron el código HTTP 201 (Creado), mientras que la petición 51 y sucesivas devolvieron un HTTP 409 (Conflicto), respetando el límite exacto . Las transacciones ACID de PostgreSQL mantuvieron la coherencia de los datos bajo estrés extremo.

9.3. Infraestructura Cloud y Despliegue Continuo (CI/CD)

El alojamiento del Frontend se delegó en la infraestructura de Vercel. Esta decisión se fundamenta en su arquitectura de "Edge Network" (CDN perimetral), que distribuye los archivos estáticos de React y Tailwind por múltiples nodos a nivel mundial . Esto garantiza que un usuario en una zona rural de Extremadura descargue la aplicación desde el nodo más cercano en milisegundos, cumpliendo con los requisitos de rendimiento.

- **Automatización mediante Webhooks:** Se configuró un *pipeline* CI/CD en el que cada comando git push origin main hacia el repositorio de GitHub notifica a Vercel . Vercel clona el código, instala dependencias, compila el proyecto y sustituye la versión antigua sin caídas del servicio (Zero Downtime Deployment).
- **Gestión Segura de Credenciales:** Para resolver la "Pantalla Blanca de la Muerte" producida en los primeros intentos de despliegue por la falta del archivo .env.local (ignorado por seguridad), las variables de entorno de Supabase se inyectaron de forma cifrada directamente en el panel de configuración *Project Settings* de Vercel .

10. CONCLUSIONES Y LÍNEAS DE EVOLUCIÓN FUTURA

El desarrollo de BicisExt culmina no solo como un ejercicio académico de ingeniería de *software*, sino como la materialización de una solución tecnológica orientada a un problema real: el vacío digital y la fragmentación en la organización del cicloturismo en Extremadura.

10.1. Conclusiones del Proyecto

Tras más de seis meses de ciclo de vida del *software*, desde la ingeniería de requisitos hasta el despliegue en producción, se extraen las siguientes conclusiones fundamentales:

- **Validación de la Arquitectura Decoupled y Serverless:** La decisión estratégica de separar la capa de presentación (Frontend con React) de la capa de datos (Supabase como BaaS) ha sido un éxito absoluto . Delegar la infraestructura del servidor ha permitido destinar el grueso de las horas de desarrollo a refinar la lógica de negocio, la experiencia de usuario (UI/UX) y la accesibilidad web.
- **Integridad y Seguridad por Diseño:** Frente a la tendencia de usar bases de datos NoSQL para prototipados rápidos, la elección de PostgreSQL ha demostrado ser vital . Las reglas de *Row Level Security* (RLS) y las restricciones de base de datos han blindado el sistema contra transacciones maliciosas y condiciones de carrera, garantizando la privacidad de los ciclistas extremeños .
- **Impacto Social (El Algoritmo de Fiabilidad):** Se ha cumplido el objetivo de crear un ecosistema seguro. BicisExt ha demostrado que, mediante un sistema de reputación matemática transparente,

es posible transformar una actividad solitaria o en grupos cerrados en una experiencia comunitaria abierta y segura, minimizando los riesgos logísticos y el absentismo .

10.2. Líneas de Evolución Futura

El estado actual de BicisExt (v1.0.0) constituye un Producto Mínimo Viable (MVP) completamente funcional e hiperlocalizado en la región de Extremadura. Sin embargo, el modelado de datos y el *stack* tecnológico han sido diseñados previendo su escalabilidad horizontal y vertical. Se plantean las siguientes vías de evolución:

1. **Desarrollo de Aplicación Móvil Nativa:** Dado que el diseño arquitectónico obedece a un modelo cliente-servidor desacoplado, la evolución natural del proyecto es el desarrollo de una *app* nativa utilizando *frameworks* como React Native o Flutter . Al estar la lógica de negocio centralizada en la API REST de Supabase, esta nueva interfaz móvil se nutriría de la base de datos actual sin necesidad de reescribir ni modificar una sola línea de código en el servidor.
2. **Procesamiento de Tracks GPS en Tiempo Real:** En la versión actual, el sistema gestiona la ubicación del punto de encuentro estático mediante la extensión espacial PostGIS. En futuras iteraciones, la infraestructura de PostgreSQL y PostGIS servirá como base matemática fundamental para permitir a los usuarios subir y procesar *tracks* GPS completos (archivos .GPX), calculando desniveles y rutas exactas sobre el mapa de forma nativa en el motor de base de datos .
3. **Escalabilidad Geográfica:** Una vez que el sistema consolide su base de usuarios activos en las provincias de Cáceres y Badajoz,

la base de datos está preparada para ampliar los filtros de la API de mapas, permitiendo que la plataforma dé el salto a comunidades autónomas limítrofes y, eventualmente, a todo el territorio nacional.

El presente documento, que recoge la Memoria Técnica y el Diario de Desarrollo de BicisExt, testifica el ciclo de vida completo de un producto de *software* desde su concepción teórica inicial hasta su despliegue integral en un entorno de producción. La realización de este proyecto no se ha limitado a la mera escritura de código, sino que ha constituido un ejercicio riguroso de ingeniería de *software*, diseño centrado en el usuario e impacto social tecnológico. A continuación, se exponen las conclusiones exhaustivas derivadas de los meses de investigación, diseño y desarrollo.

11 Impacto Social y Resolución del Problema Base (El Vacío Digital)

El origen de BicisExt nació de una necesidad real detectada en la Comunidad Autónoma de Extremadura. El análisis inicial del mercado reveló que, si bien la región cuenta con una de las redes de caminos y rutas paisajísticas más ricas de España , existía una profunda fragmentación digital en la comunidad ciclista. Las plataformas globales dominantes, como Strava o Komoot, se enfocan casi exclusivamente en

el rendimiento físico individual y el descubrimiento asíncrono de rutas, ignorando por completo la logística humana de los encuentros presenciales. Por otro lado, la dependencia de grupos de mensajería instantánea generaba caos informativo y desconfianza.

BicisExt ha logrado mitigar este problema transformando una actividad individual en una experiencia colectiva segura. La implementación del "Algoritmo de Fiabilidad" y el "Radar de Reputación" ha demostrado que es posible construir un ecosistema de confianza cruzada. Al obligar a los usuarios a definir parámetros estrictos como el ritmo medio y el tipo de terreno (Carretera, MTB, Gravel) mediante enumeraciones de base de datos (ENUM) , se ha erradicado el "Síndrome del Desajuste de Nivel", garantizando salidas seguras y acordes a las capacidades físicas de cada participante.

11.1 Éxito de la Arquitectura Desacoplada y el Ecosistema Serverless

A nivel arquitectónico, la decisión de huir de los sistemas monolíticos tradicionales para abrazar el paradigma "JAMstack" ha sido el pilar del éxito técnico del proyecto. La separación total entre la capa de presentación (Frontend) y la capa de lógica y persistencia de datos (Backend) garantiza una escalabilidad futura sin precedentes; si el día

de mañana se decide desarrollar una aplicación móvil nativa (con React Native o Flutter), la API y la base de datos actuales servirían sin necesidad de refactorizar el código del servidor .

La elección del *Stack* Tecnológico ha demostrado ser óptima para los recursos de un desarrollador individual:

- **Frontend Reactivo:** El uso de React (v18+) junto con Vite ha permitido crear una *Single Page Application* (SPA) extremadamente fluida . La recarga instantánea de componentes mediante el *Virtual DOM* evita la fatiga del usuario y simula la experiencia de una aplicación nativa instalada en el dispositivo móvil .
- **Backend as a Service (BaaS):** Delegar la infraestructura a Supabase fue la decisión más crítica y acertada . Frente a opciones NoSQL como Firebase, que hubieran dificultado las complejas relaciones entre usuarios y valoraciones, el motor relacional de PostgreSQL proporcionó la robustez necesaria . Además, la integración nativa de la extensión geoespacial PostGIS permitió realizar cálculos de distancias teniendo en cuenta la curvatura real de la Tierra, superando las limitaciones matemáticas básicas .

11.2 Seguridad por Diseño y Resiliencia del Sistema

El desarrollo de BicisExt no ha asumido en ningún momento el "Camino Feliz" (*Happy Path*). Desde la primera línea de código, el sistema fue diseñado bajo la premisa de que los usuarios cometen errores o pueden tener intenciones maliciosas .

- **Seguridad a Nivel de Datos:** La implementación de políticas de seguridad a nivel de fila (*Row Level Security* - RLS) en PostgreSQL ha blindado los datos de los ciclistas. Instrucciones a nivel de *kernel* como auth.uid() = creator_id garantizan que es matemáticamente imposible que un usuario modifique, altere o borre rutas que no le pertenecen, sin importar los fallos que pudieran existir en el Frontend . Asimismo, el uso de identificadores UUID v4 erradicó de raíz las vulnerabilidades IDOR (*Insecure Direct Object Reference*), protegiendo la plataforma contra el raspado de datos masivo (*Scraping*) .
- **Integridad Transaccional bajo Estrés:** Las pruebas de carga (*Load Testing*) validaron el esfuerzo en la normalización de la base de datos hasta la Tercera Forma Normal (3FN). Al someter el sistema a 1.000 peticiones concurrentes para un aforo de 50 plazas, las restricciones lógicas y de unicidad (UNIQUE CONSTRAINT) de PostgreSQL previnieron cualquier condición de carrera (*Race Condition*), devolviendo exactamente 50 códigos HTTP 201 (Éxito) y bloqueando el resto con códigos HTTP 409 (Conflicto) .

11.3 Diseño Inclusivo, Rendimiento y Accesibilidad

BicisExt se concibió bajo una filosofía *Mobile-First*, asumiendo que el 90% de sus interacciones ocurrirían en exteriores, bajo la luz directa del sol y mediante conexiones inestables en zonas rurales de Extremadura.

Para cumplir con estos estrictos Requisitos No Funcionales (RNF), se combinaron varias estrategias:

- **Identidad Visual y Contraste:** Se rechazó el "Modo Oscuro" por defecto, adoptando una paleta basada en "Blanco Humo" y "Verde Dehesa" que asegura un contraste absoluto y cumple con los estándares de lectura . El uso de Tailwind CSS garantizó un archivo de estilos minificado de apenas unos kilobytes .
- **Accesibilidad (WCAG 2.1):** El código HTML se reestructuró con rigor semántico . La inclusión de atributos aria-label, la corrección de ratios de contraste cromático y la implementación de anillos de enfoque (focus-visible) garantizan que ciclistas con visión reducida o daltonismo puedan operar la plataforma mediante lectores de pantalla o navegación por teclado de forma íntegra .
- **Despliegue Continuo Perimetral:** El uso de los nodos CDN (*Edge Network*) de Vercel para el despliegue automático CI/CD aseguró que los tiempos de renderizado inicial (FCP) cayeran por debajo de los 1.5 segundos exigidos, democratizando el acceso a la plataforma independientemente de la calidad de cobertura 4G del usuario .

11.4 Crecimiento Metodológico y Valoración Personal

A nivel formativo, el desarrollo de este proyecto aplicando el marco de trabajo SCRUM y la división del esfuerzo en *Sprints* semanales ha sido fundamental para gestionar la complejidad de un *software* de tal

magnitud en solitario . La obligación de documentar constantemente la deuda técnica, refactorizar componentes extensos y mantener un control de versiones estricto con Git ha forjado una disciplina que trasciende la simple programación académica, acercándose a las exigencias operativas de un entorno empresarial *Senior*.

BicisExt ha dejado de ser una simple idea en un papel para convertirse en un Producto Mínimo Viable (MVP) robusto, documentado, asegurado y desplegado en la nube. Es una herramienta tecnológica concebida en Extremadura y para Extremadura, con un modelo de datos preparado para escalar a nivel nacional. Este proyecto demuestra que el *software* verdaderamente útil es aquel que, apoyándose en la ingeniería más avanzada, consigue devolver a las personas el valor de la conexión humana.